The Systems Behind Modern AI

Why and How Hardware Designed for 3D Graphics Powers Modern AI

Sam Ly

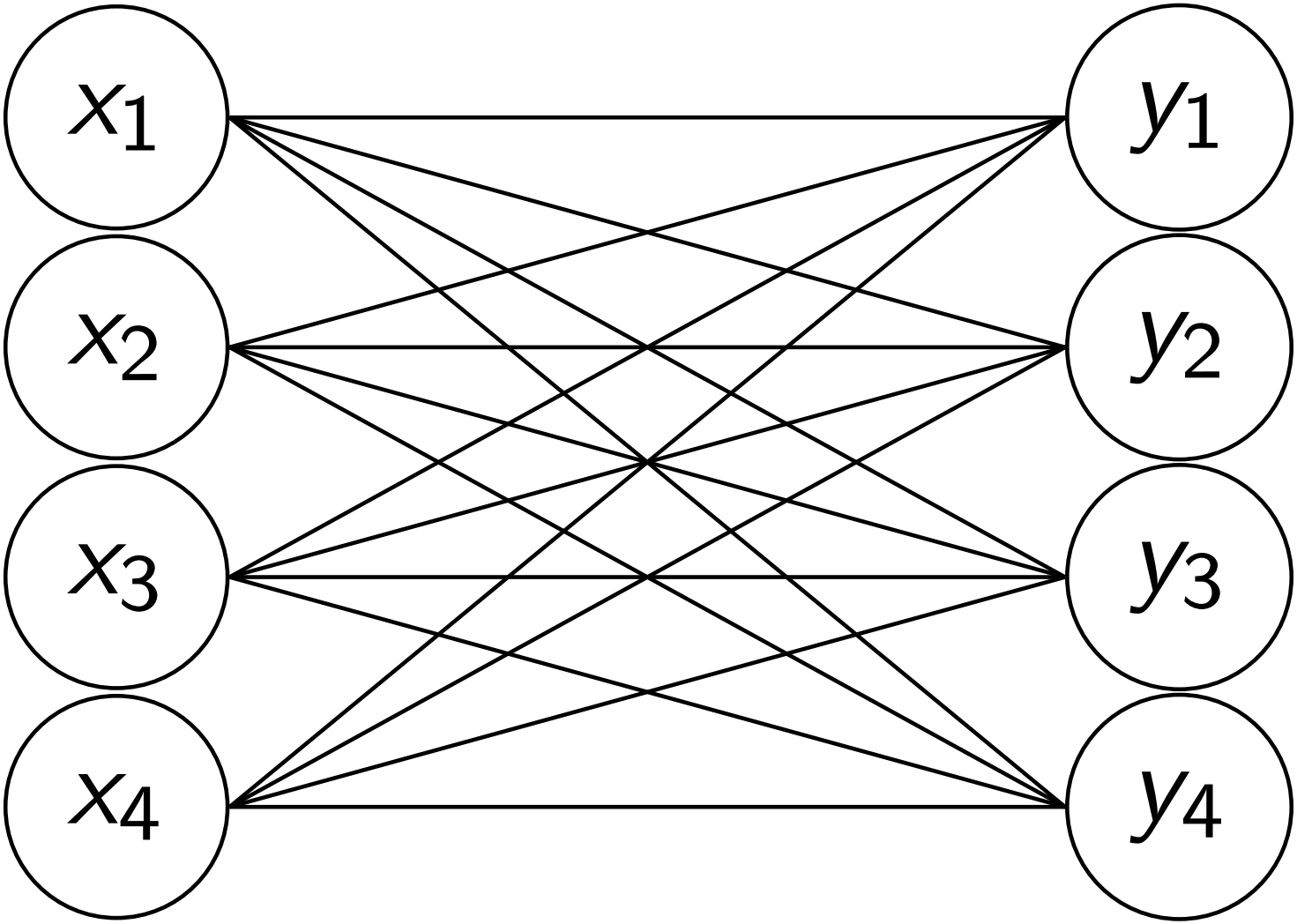
Cal Poly Pomona

Fall 2026

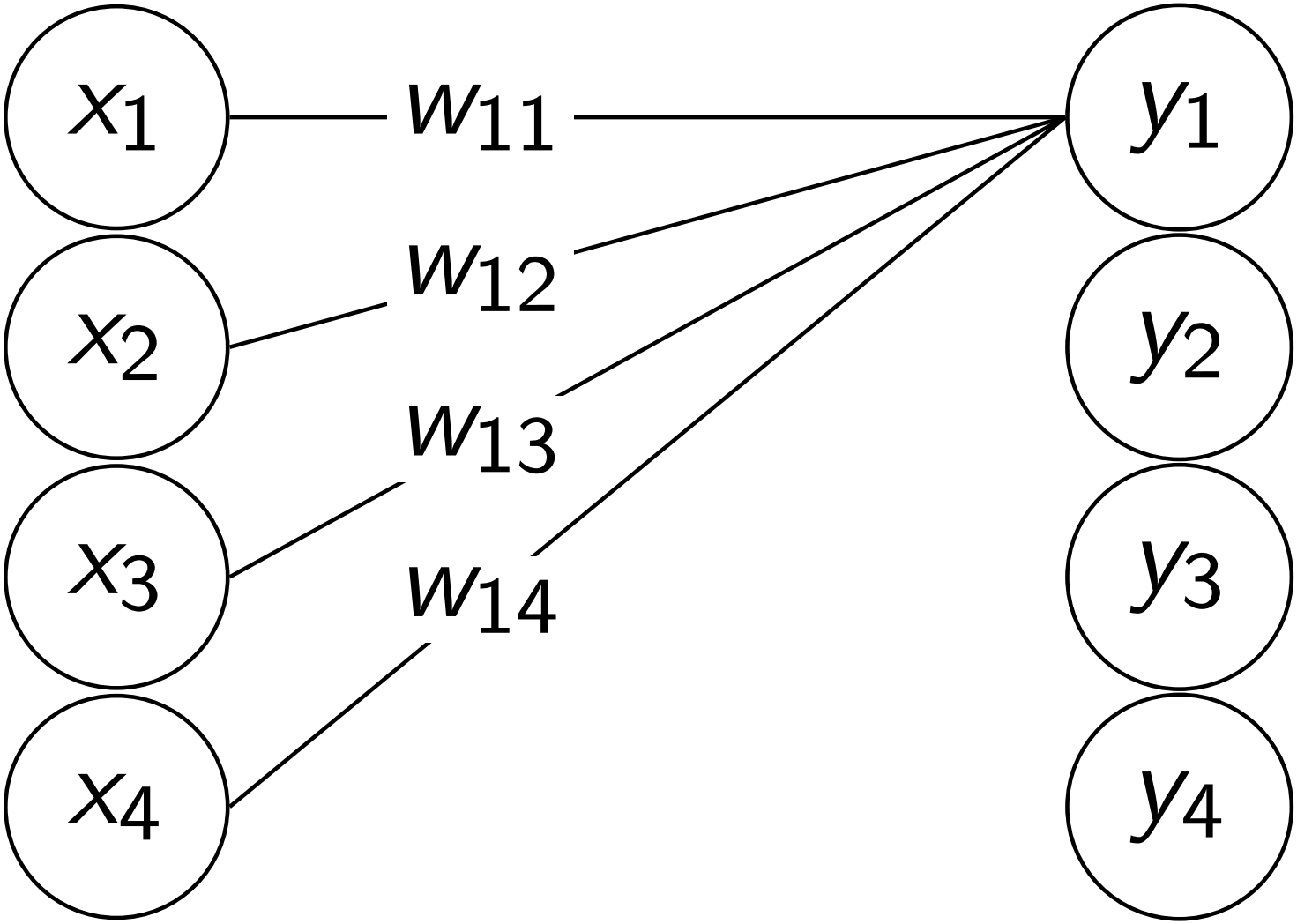
Table of Contents

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

Basic Neural Net (Linear Layer)

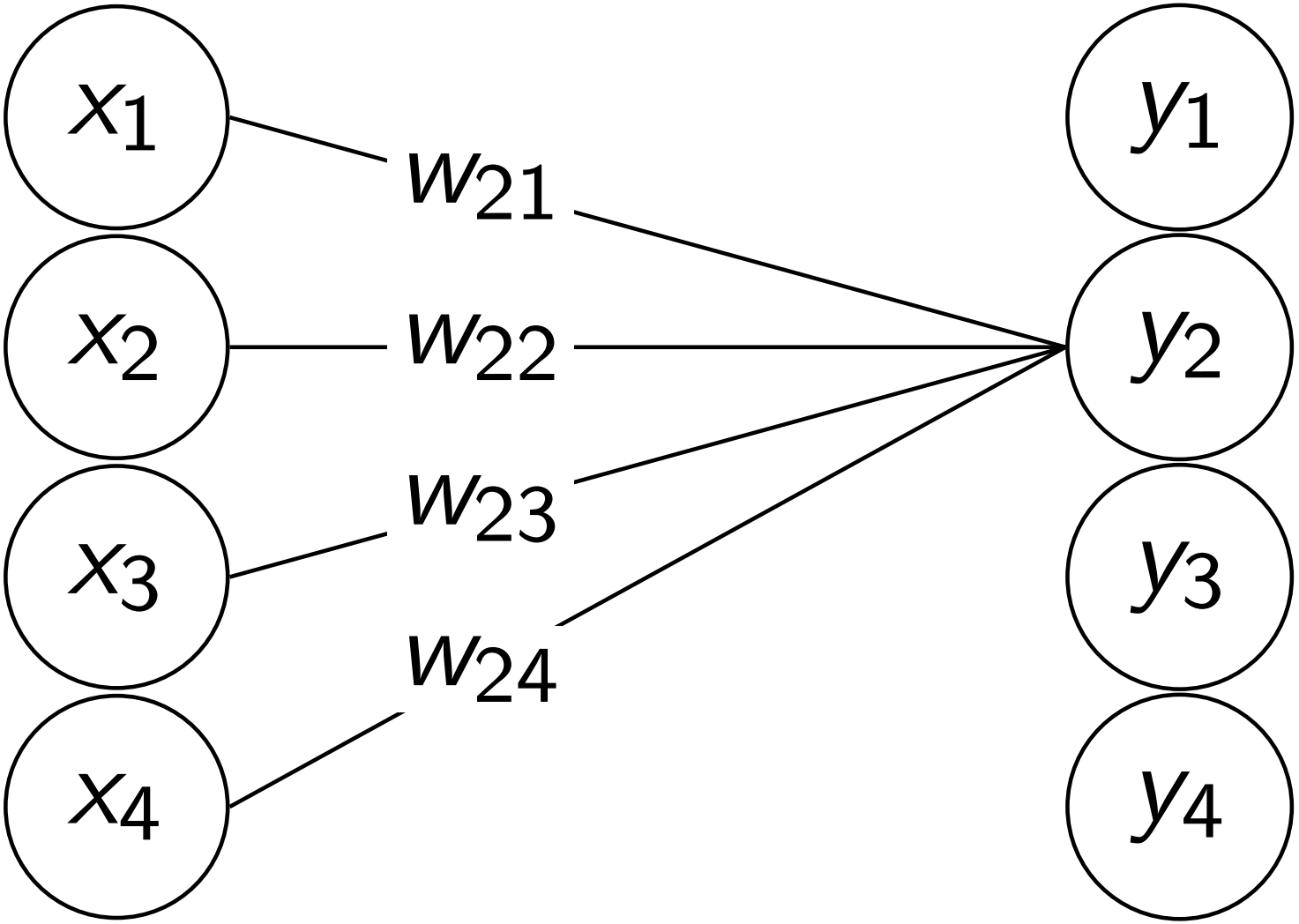


Basic Neural Net (Linear Layer)



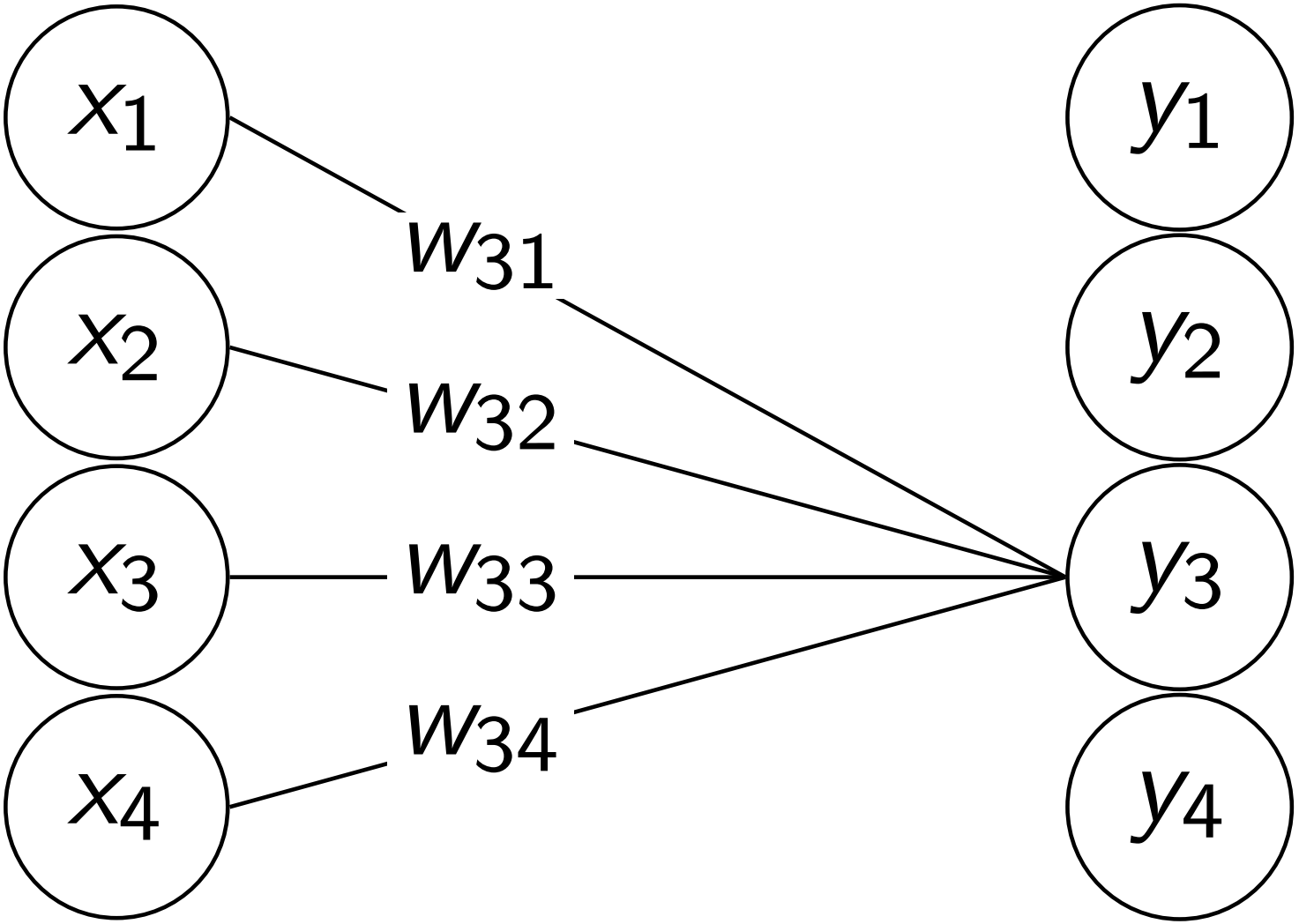
$$y_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14}$$

Basic Neural Net (Linear Layer)



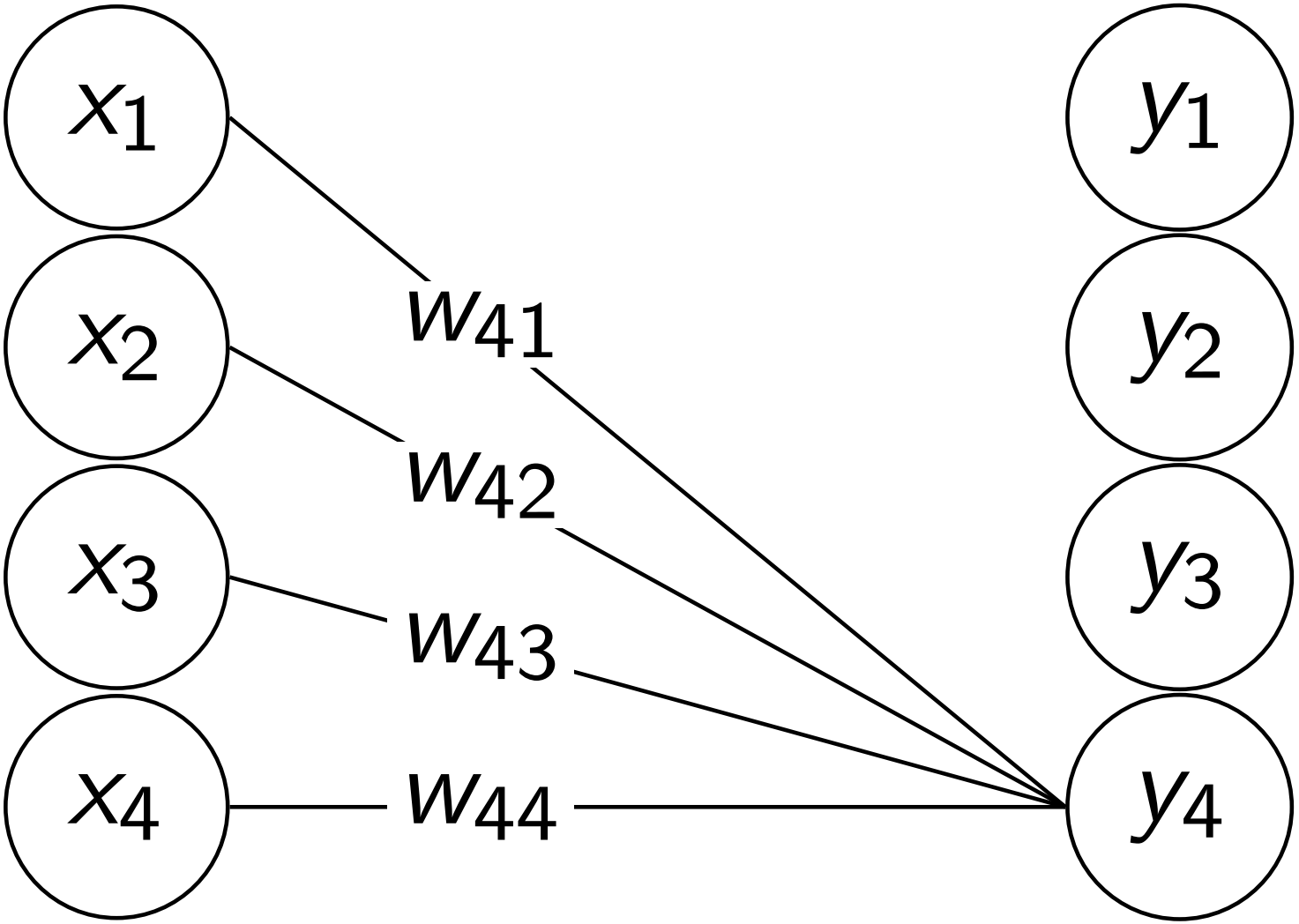
$$y_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24}$$

Basic Neural Net (Linear Layer)



$$y_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34}$$

Basic Neural Net (Linear Layer)



$$y_4 = x_1 w_{41} + x_2 w_{42} + x_3 w_{43} + x_4 w_{44}$$

Linear Algebra Interpretation

Here, notice that for each y_i, we are taking a weighted sum of the x elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

Linear Algebra Interpretation

Here, notice that for each y_i, we are taking a weighted sum of the x elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

Arranging the y_i into a vector, we get

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

Linear Algebra Interpretation

We can generalize the number of x's and the number of y's.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

Linear Algebra Interpretation

We can generalize the number of x's and the number of y's.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

We say, n is the *input dimension* and m is the *output dimension*.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

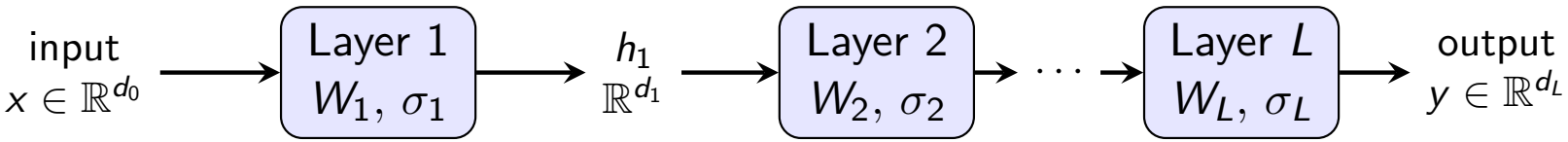
We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

New Representation

We have successfully mapped the batched computation of a **linear layer** a matrix multiplication!

Composing Layers

We can view W as a function that takes in an n-dimensional vector and maps it to an m-dimensional vector. $W : \mathbb{R}^n \to \mathbb{R}^m$. We create deep neural networks by *composing* these transformations, along with **nonlinearities** σ_i.



$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

Takeaways

1. A linear layer applied to a batch of inputs can be expressed as a single matrix multiplication, $Y = WX$.
2. Deep networks are composed of many of these linear transformations.
3. **Matrix multiplication therefore captures a large and reusable part of neural network computation.**

Why is this representation useful?

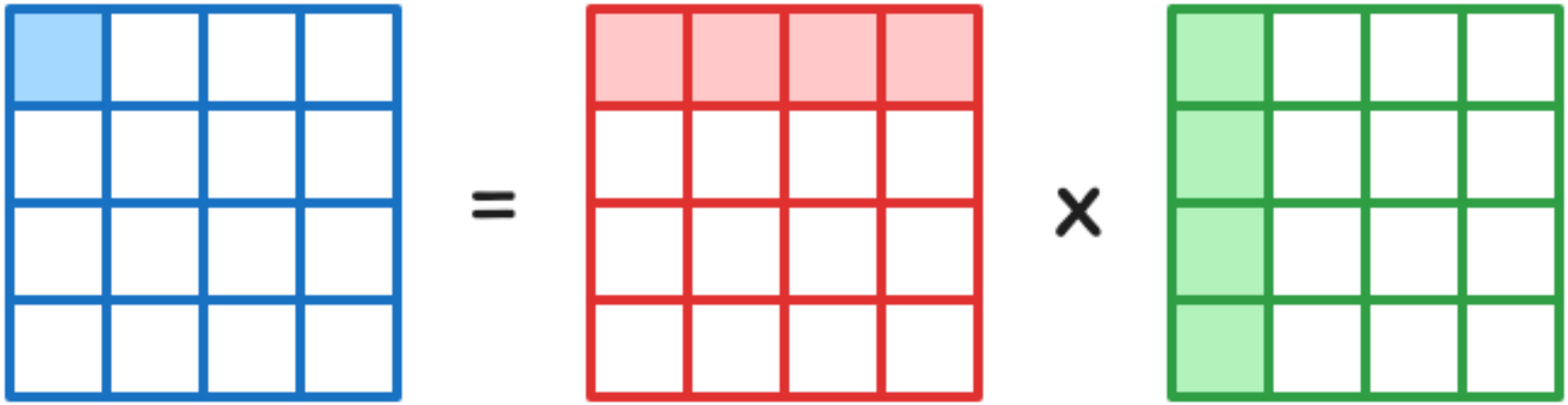
By optimizing a the matrix multiplication, we also optimize neural networks.

Table of Contents

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

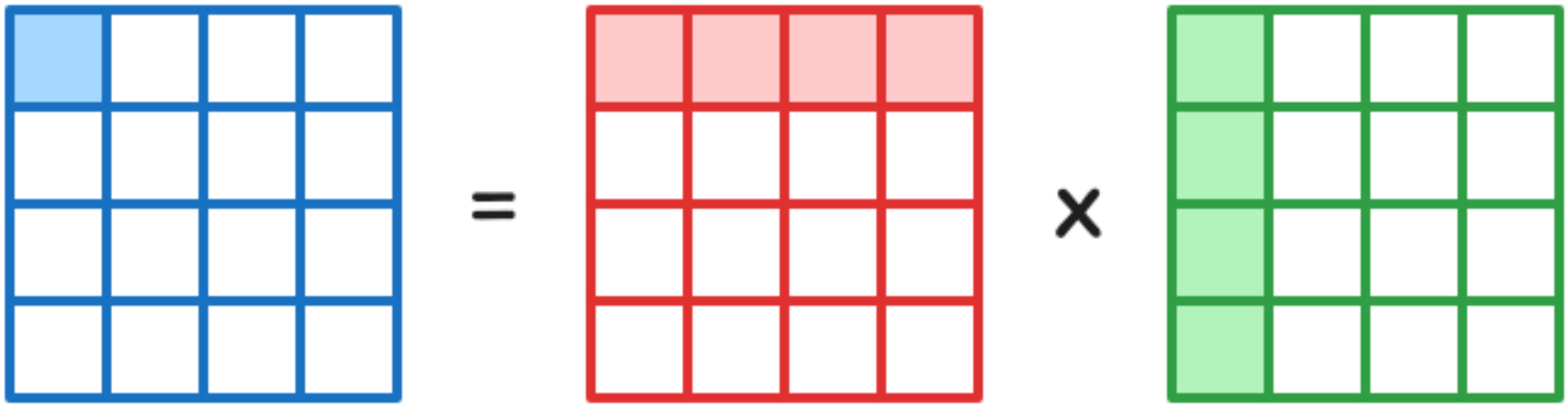
Computing a Matrix Multiplication

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



Computing a Matrix Multiplication

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



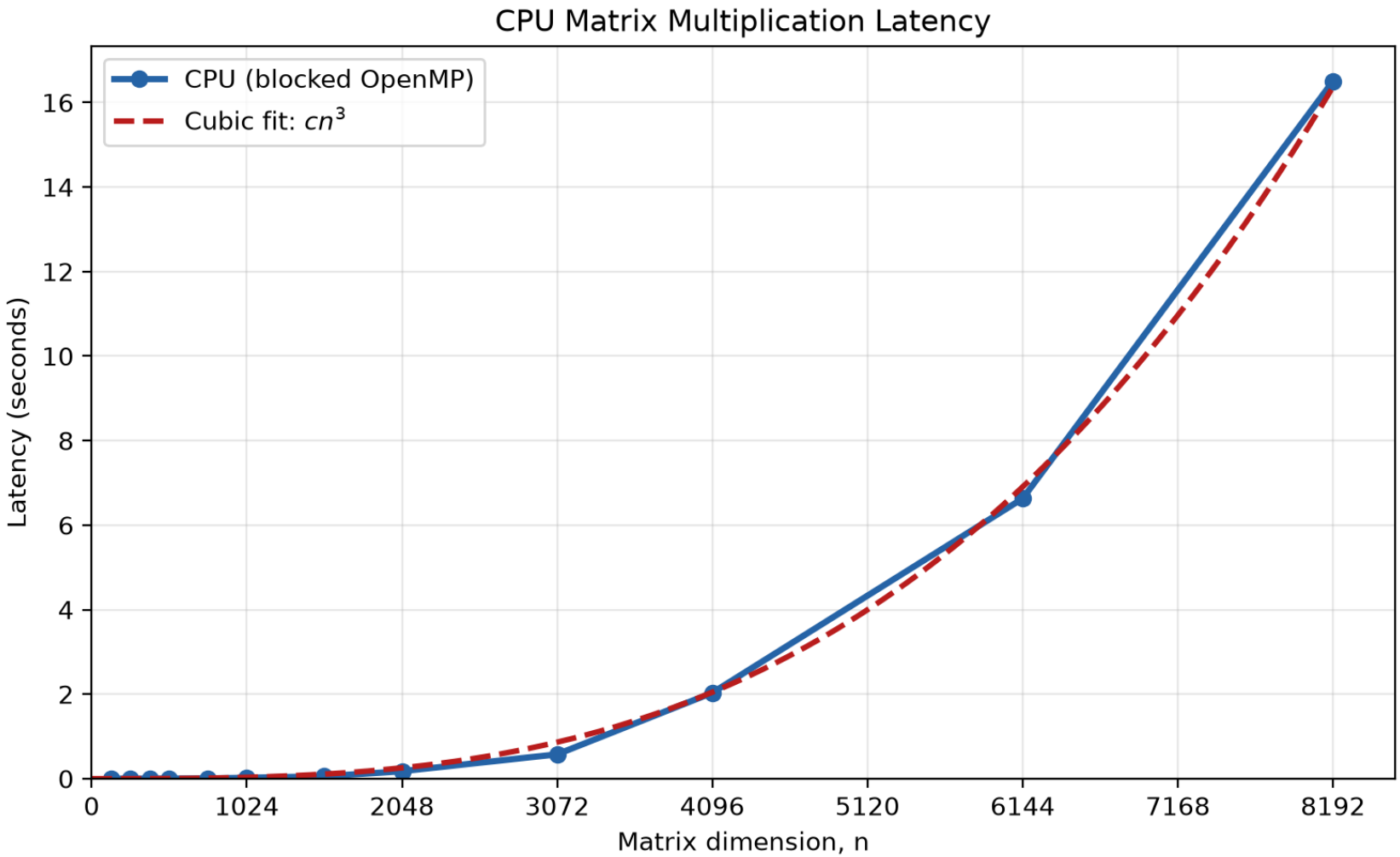
Poor scaling

The operations are very simple, but the number of operations grow on the order of $\mathcal{O}(n^3)$, where n is the dimensions of the matrices.

Runtime of a matrix multiplication on a CPU

```
for (int y = 0; y < n; ++y) {
  for (int x = 0; x < n; ++x) {
    double sum = 0.0;
    for (int k = 0; k < n; ++k) {
      sum += a[y*n+k] * b[k*n+x];
    }
    c[y*n+x] = sum;
  }
}
```

Runtime of a matrix multiplication on a CPU



What are GPUs for?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

What are GPUs for?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

Graphics exposes data parallelism

Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

What are GPUs for?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

Graphics exposes data parallelism

Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

A CPU uses a small number of sophisticated cores to minimize the latency of one instruction stream. A GPU instead uses many simpler execution units to maximize *throughput*.

What are GPUs for?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

Graphics exposes data parallelism

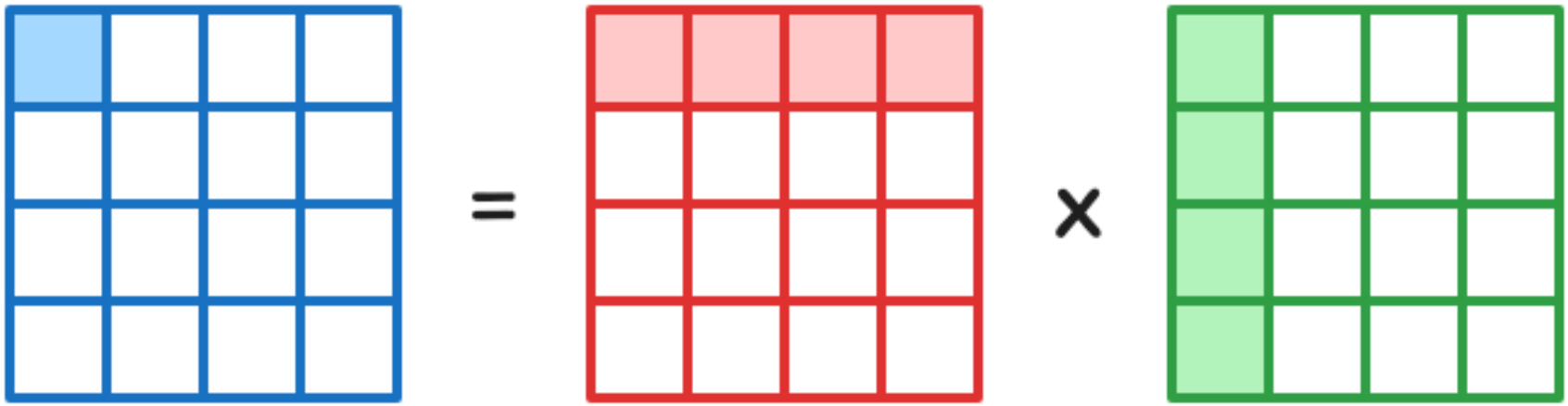
Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

A CPU uses a small number of sophisticated cores to minimize the latency of one instruction stream. A GPU instead uses many simpler execution units to maximize *throughput*.

The key tradeoff

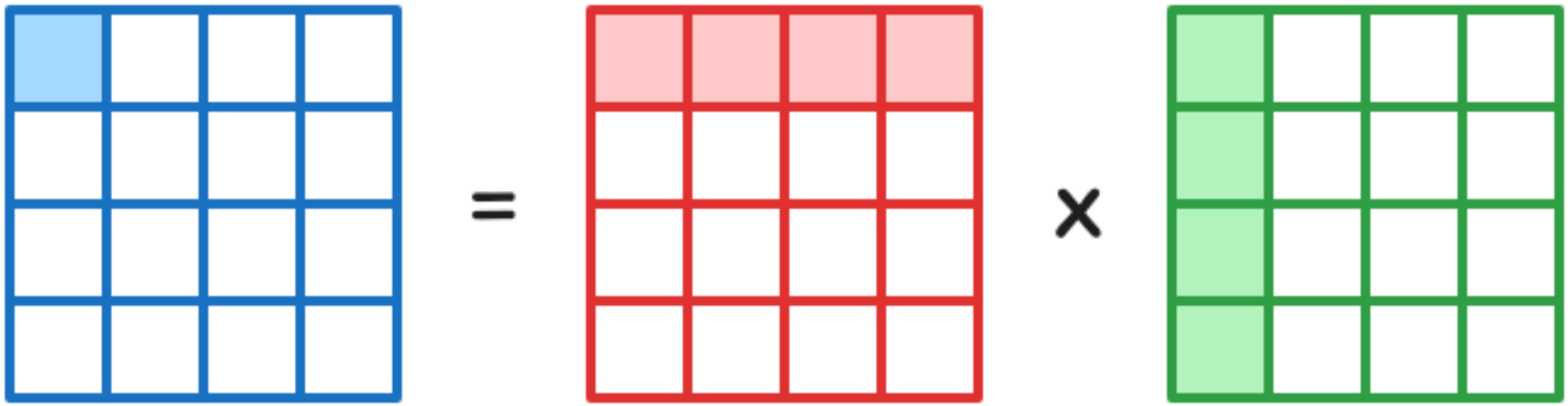
Graphics rendering is difficult because of the **volume** of similar computations, not because any one computation is especially complex.

Matrix Multiplication is Parallel



Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Matrix Multiplication is Parallel



Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Key Idea

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication?

Takeaway

- CPUs can't efficiently perform matrix multiplication at scale.
- Graphics rendering is a parallel task, and so is matrix multiplication. Thus, GPUs can be good at matrix multiplication.

Whats next

I've suggested that GPUs *can* do matrix multiplication, but how do we actually get there?

Table of Contents

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

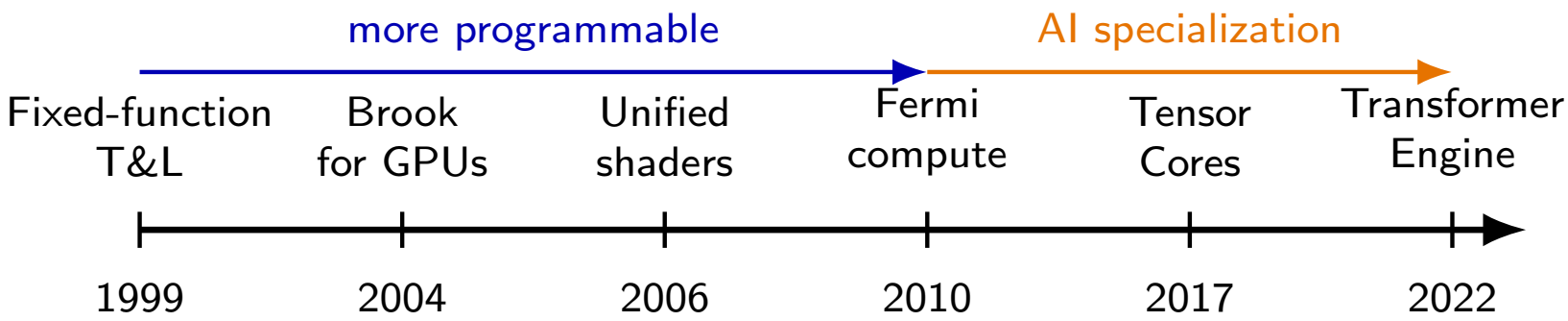
Timeline

- **1999—2003:** fixed-function graphics gives way to programmable vertex and pixel shaders.
- **2004:** Brook for GPUs demonstrates a high-level stream-programming model on top of graphics APIs.
- **2006—2007:** GeForce 8800 introduces a unified shader architecture; NVIDIA releases CUDA publicly in 2007.
- **2010:** Fermi adds a coherent cache hierarchy, ECC support, and much stronger support for C++ and scientific computing.
- **2017:** Volta introduces Tensor Cores for mixed-precision matrix multiply–accumulate.
- **2020—2024:** Ampere broadens Tensor Core formats; Hopper adds the FP8 Transformer Engine; Blackwell adds further low-precision and AI-specific hardware.

What shaped modern GPU architecture?

Driver of architectural change

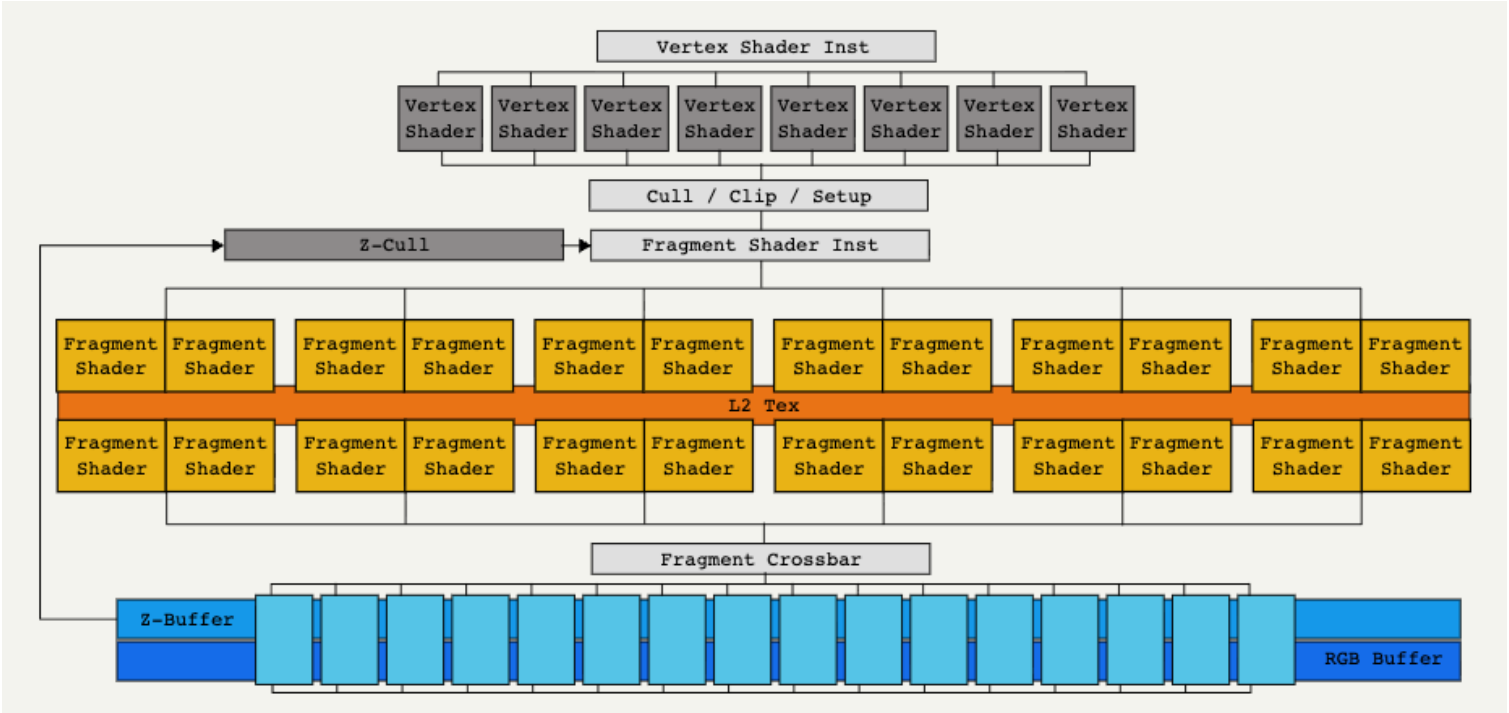
New workloads cause generalization/programmability, established workloads cause specialization.



What shaped modern GPU architecture?

<2006: graphics first

- GeForce 256 (1999) accelerated fixed-function transform and lighting.
- Later chips added programmable vertex and pixel shaders, but kept distinct stages and resources.



The G71 architecture (2005). Diagram source: Sanglard (2020).

What shaped modern GPU architecture?

2004: GPGPU before CUDA. Demand was always there.

- Brook for GPUs compiled a stream language to graphics APIs.
- It showed useful non-graphics workloads could exploit GPU throughput, despite an awkward programming model.

Brook for GPUs: Stream Computing on Graphics Hardware

Ian Buck Tim Foley Daniel Horn Jeremy Sugerman Kayvon Fatahalian Mike Houston Pat Hanrahan
Stanford University

Abstract

In this paper, we present Brook for GPUs, a system for general-purpose computation on programmable graphics hardware. Brook extends C to include simple data-parallel constructs, enabling the use of the GPU as a streaming co-processor. We present a compiler and runtime system that abstracts and virtualizes many aspects of graphics hardware. In addition, we present an analysis of the effectiveness of the GPU as a compute engine compared to the CPU, to determine when the GPU can outperform the CPU for a particular algorithm. We evaluate our system with five applications, the SAXPY and SGEMV BLAS operators, image segmentation, FFT, and ray tracing. For these applications, we demonstrate that our Brook implementations perform comparably to hand-written GPU code and up to seven times faster than their CPU counterparts.

CR Categories: I.3.1 [Computer Graphics]: Hardware Architecture—Graphics processors D.3.2 [Programming Languages]: Language Classifications—Parallel Languages

Keywords: Programmable Graphics Hardware, Data Parallel Computing, Stream Computing, GPU Computing, Brook

modern hardware. In addition, the user is forced to express their algorithm in terms of graphics primitives, such as textures and triangles. As a result, general-purpose GPU computing is limited to only the most advanced graphics developers.

This paper presents *Brook*, a programming environment that provides developers with a view of the GPU as a streaming coprocessor. The main contributions of this paper are:

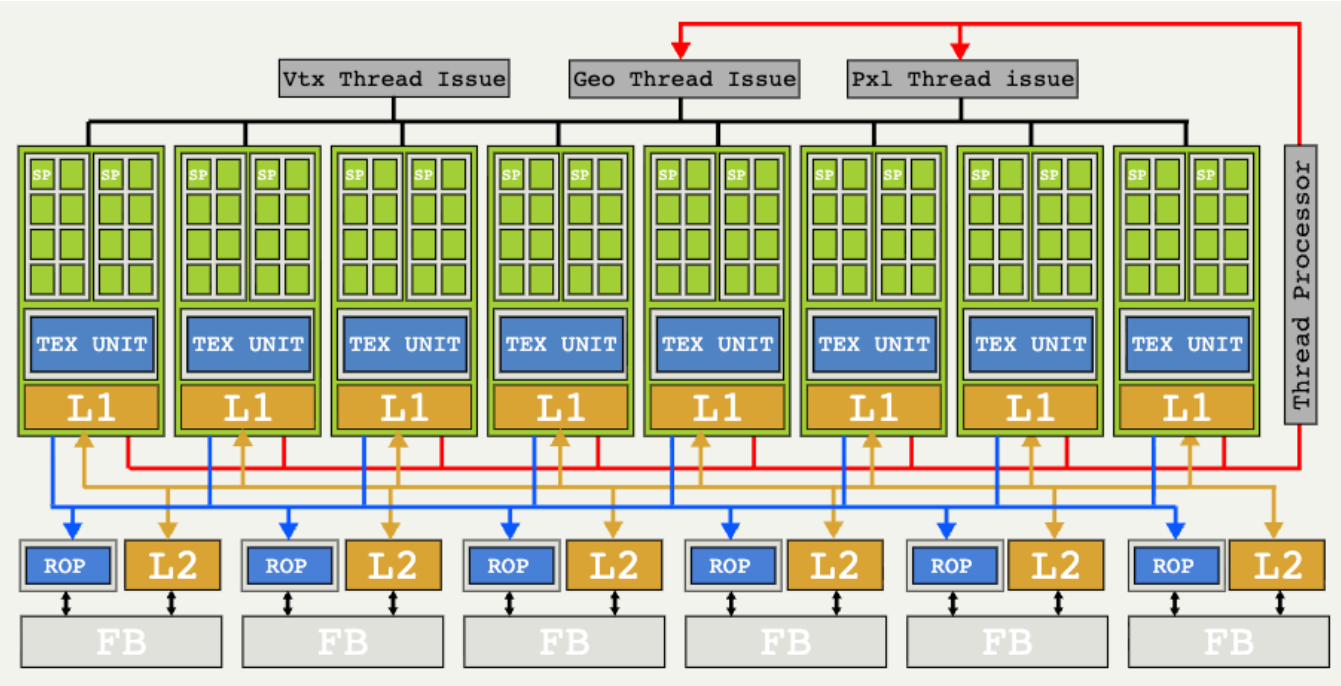
- The presentation of the Brook stream programming model for general-purpose GPU computing. Through the use of streams, kernels and reduction operators, Brook abstracts the GPU as a streaming processor.
- The demonstration of how various GPU hardware limitations can be virtualized or extended using our compiler and runtime system; specifically, the GPU memory system, the number of supported shader outputs, and support for user-defined data structures.
- The presentation of a cost model for comparing GPU vs. CPU performance tradeoffs to better understand under what circumstances the GPU outperforms the CPU.

Buck et al., "Brook for GPUs," *ACM TOG* 23(3), 2004.

What shaped modern GPU architecture?

2006-2007: increasing complexity necessitates unified compute

- G80 ran vertex, geometry, and pixel work on unified pool of **Streaming Multiprocessors** (SM's).
- **This is the heart of modern GPU architecture.**



G80 block diagram. Diagram source: Sanglard (2020); architecture described by Lindholm et al. (2008).

Our model of the GPU (Execution)

The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

Our model of the GPU (Execution)

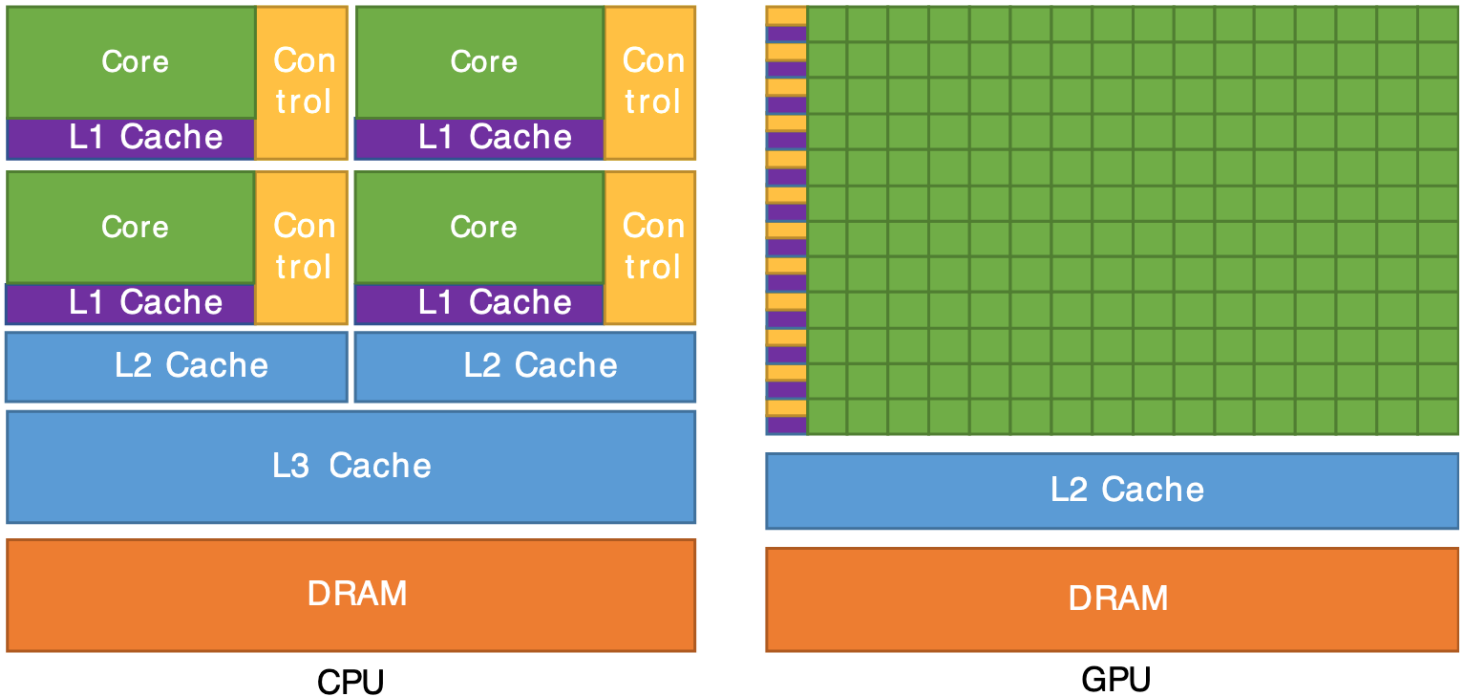
The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

The CUDA Cores execute the same instruction in parallel on different data.

Our model of the GPU (Execution)

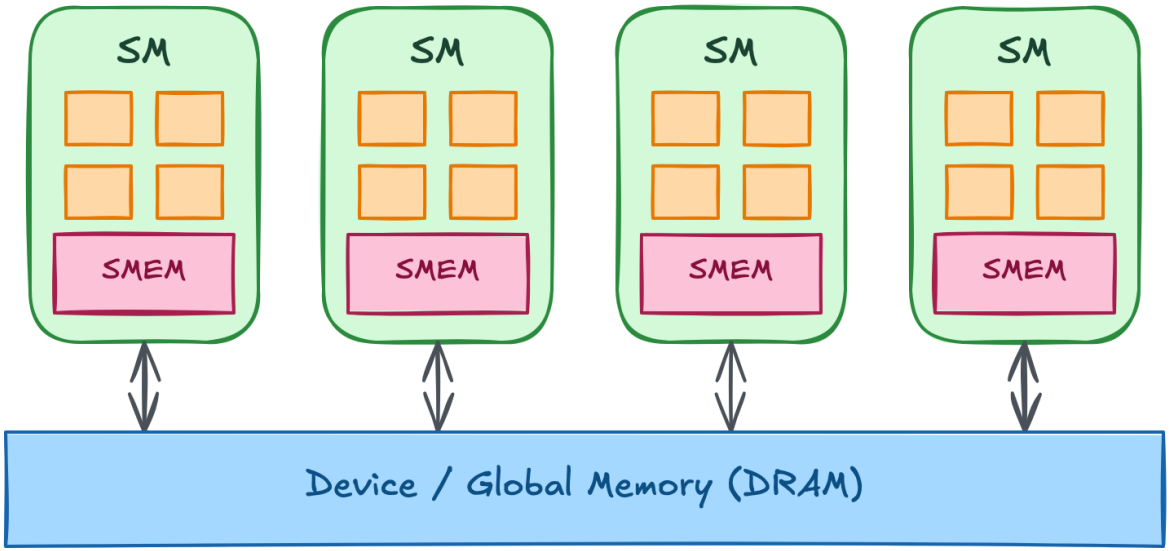
The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

The CUDA Cores execute the same instruction in parallel on different data.



Our model of the GPU (Memory)

The GPU has its own memory called **Device Memory (DRAM)**. Each SM has a **programmable L1 Cache** which we also call **Shared Memory (SMEM)**.

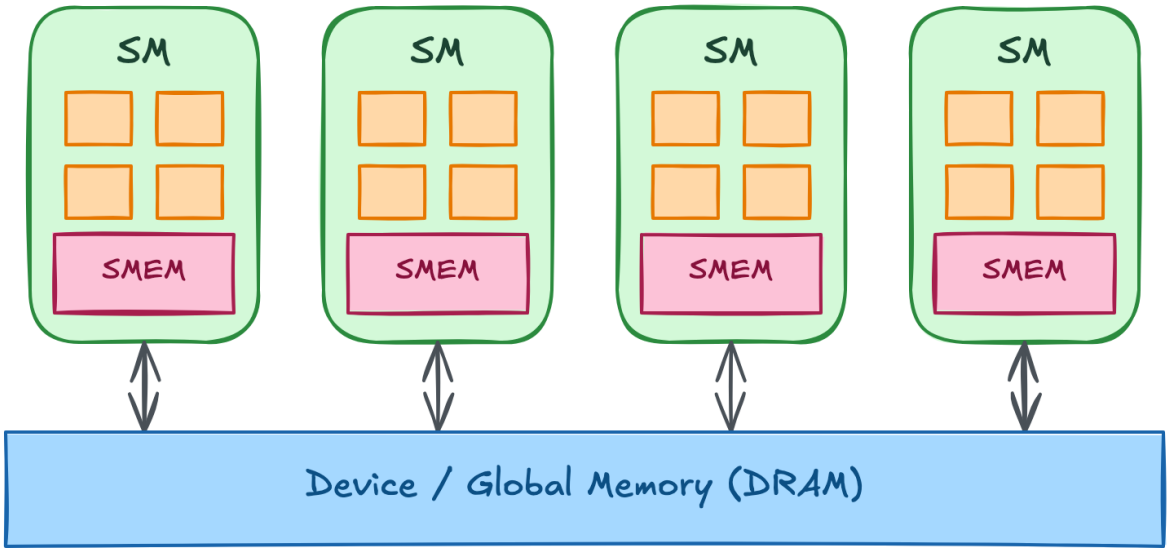


Performance

Accessing to DRAM is orders of **tens of times** slower than accessing SMEM. Effective use of SMEM is crucial to performance.

Takeaways

- SMs: primary compute unit
- CUDA Cores: like the "ALUs" of the SMs.
- L1 cache/Shared memory: cache for the SM
- Device/Global Memory: like RAM for a normal CPU system.



Whats next

How can we actually implement a matrix multiplication on a GPU? What tools and mental models do we need?

Table of Contents

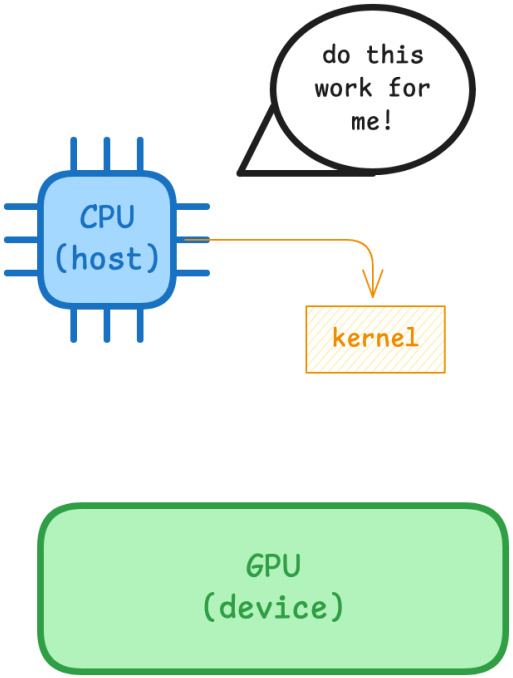
1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

Section Overview

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

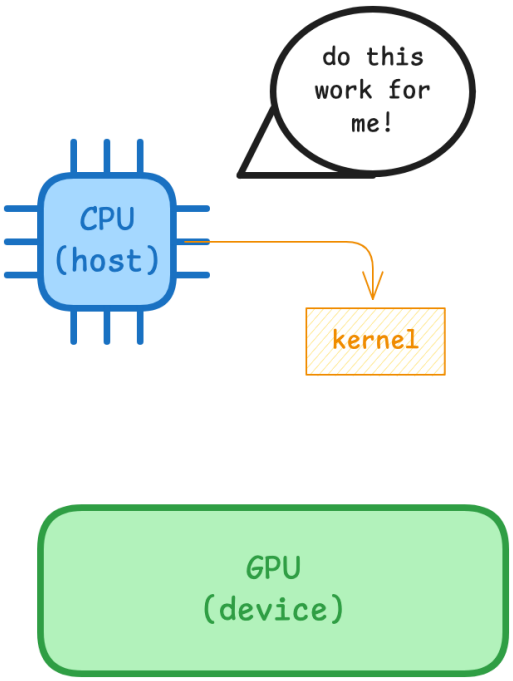
CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.



CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.




```
int main() {
  const int M = 128, N = 128, K = 128;

  float *dA, *dB, *dC;
  cudaMalloc(&dA, M * K * sizeof(float);
  cudaMalloc(&dB, K * N * sizeof(float));
  cudaMalloc(&dC, M * N * sizeof(float));

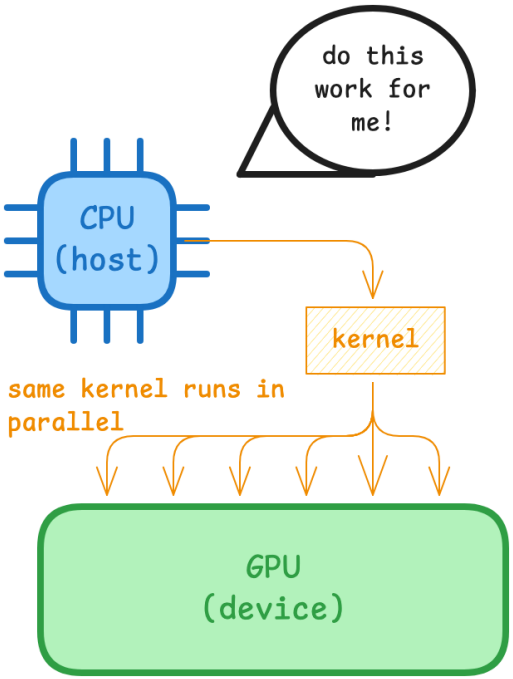
  // load values into these device pointers...

  run_sgemm_naive(M, N, K, 1, dA, dB, 0, dC);

  cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
```

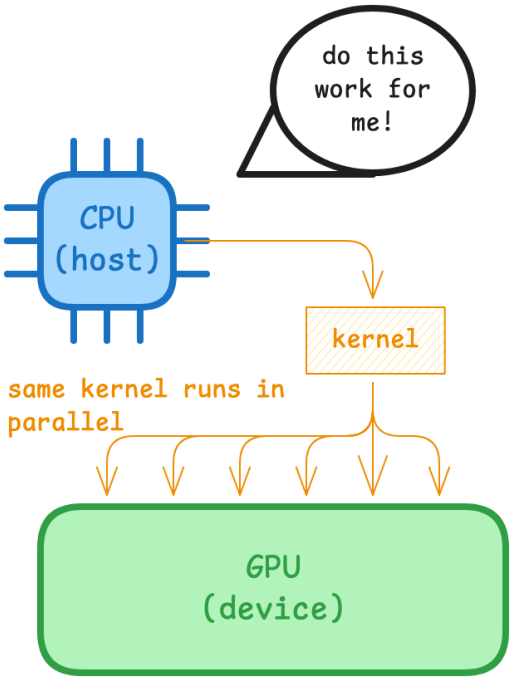
CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.



CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.

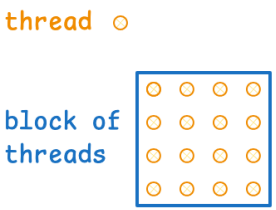


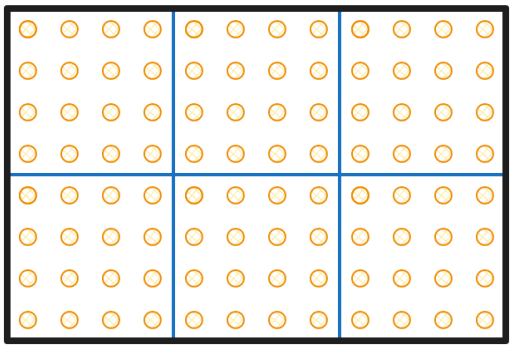

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}
```

```
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

CUDA Basics (Threads, Blocks, Grids)

- Threads are grouped into **blocks**.
- The collection of blocks requested by the host is organized in a **grid**.






```cpp
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}
```

```cpp
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Relation to hardware

This hierarchy is closely related to how the device schedules the execution. We will see later how to use this to optimize performance.

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Relation to hardware

This hierarchy is closely related to how the device schedules the execution.
We will see later how to use this to optimize performance.

Thread, Block, Grid hierarchy

Threads are grouped into blocks of constant size/shape.
Blocks are launched as a grid.
We choose the dimensions of the grid to control how many blocks are launched, and thus how many threads are launched.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

Example

During the forward pass when training a model, we accumulate the gradients from the entire batch.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

```
sgemm(
  int M, int N, int K,
  float alpha, float *A, float *B,
  float beta, float *C
)
```

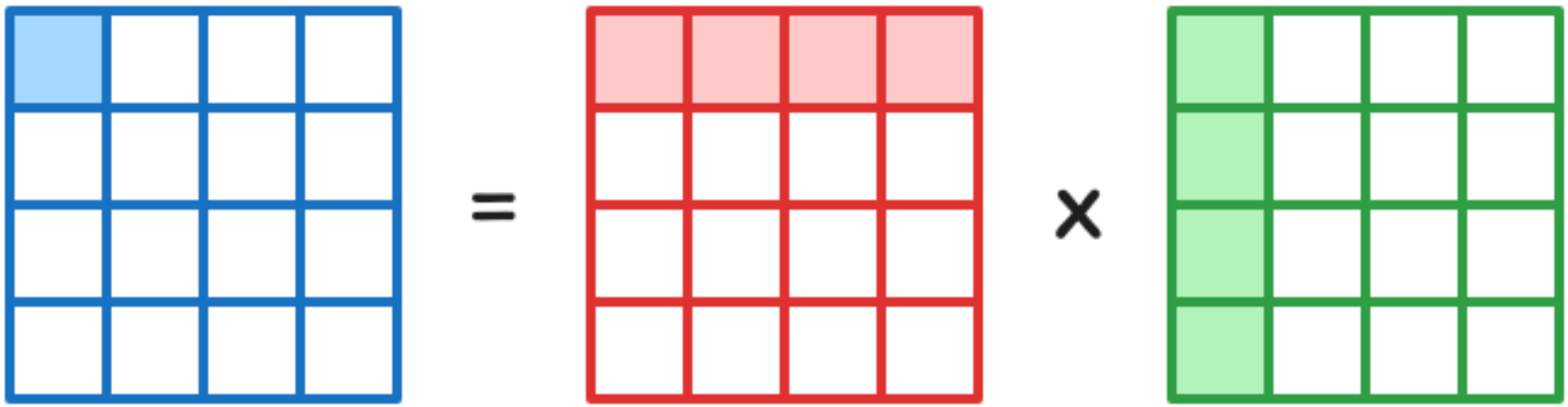
$$\underbrace{A}_{M \times K} \quad \underbrace{B}_{K \times N} \quad \underbrace{C}_{M \times N}$$

- M: rows of A and C
- N: columns of B and C
- K: shared reduction dimension

Let's design a SGEMM kernel!

Recall

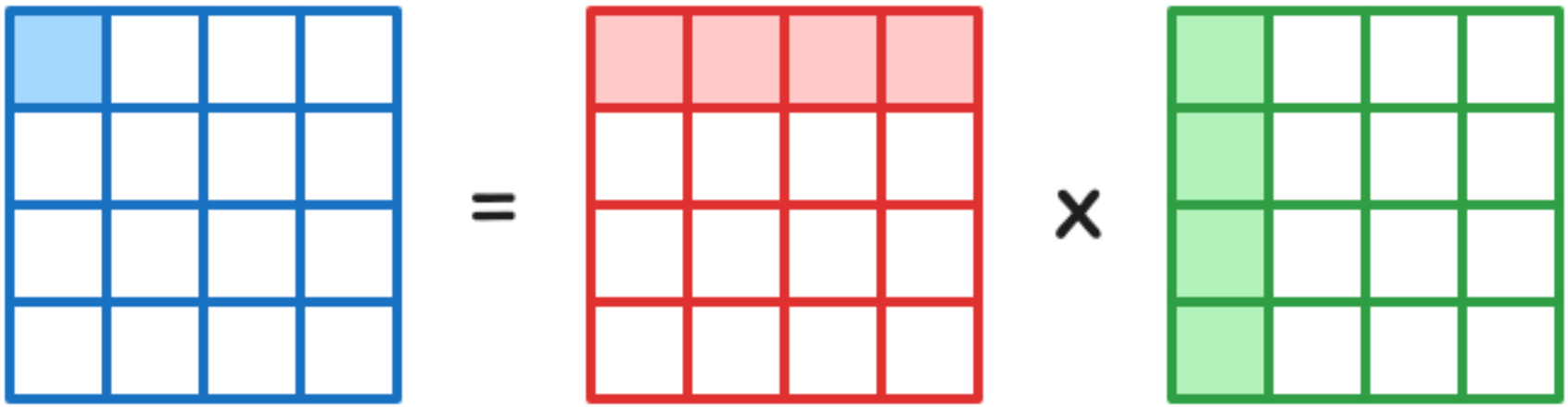
Recall that matrix multiplication is naturally parallel, and the output elements of C can be computed independently in parallel.



Let's design a SGEMM kernel!

Recall

Recall that matrix multiplication is naturally parallel, and the output elements of C can be computed independently in parallel.



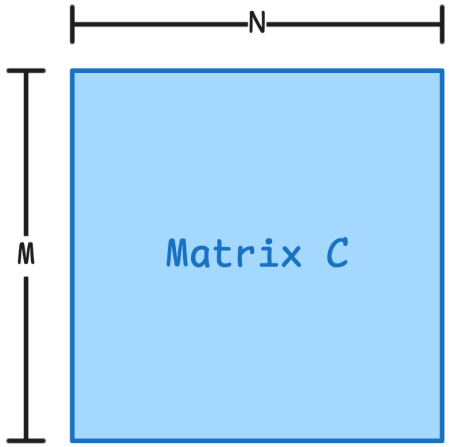
We can naturally map one thread to each output element!

Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

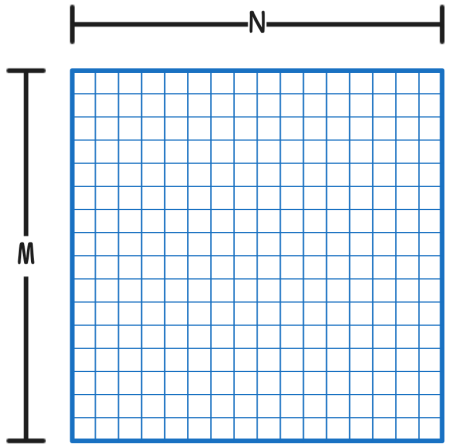


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

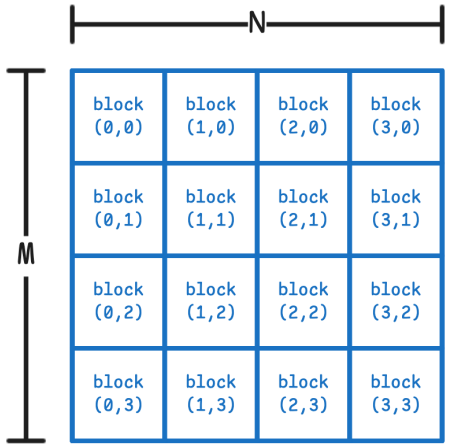


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

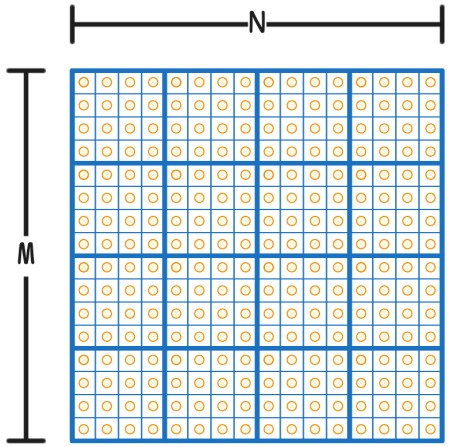


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

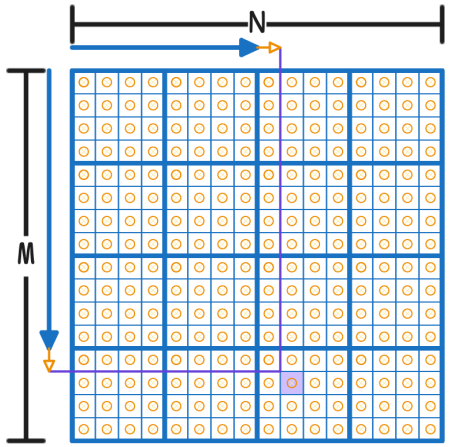


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```



Naive SGEMM visualization

- One thread computes one element of C.
- It walks one row of A and one column of B for the full dot product.

Play naive-kernel visualization

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C.

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000.

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000.

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
int M = 5000, N = 4000, K = 3000;
float alpha = 1, beta = 0;

float *dA, *dB, *dC; // Allocate buffers on the DEVICE.
cudaMalloc(&dA, M * K * sizeof(float);
cudaMalloc(&dB, K * N * sizeof(float));
cudaMalloc(&dC, M * N * sizeof(float));
// BM, BN are compile-time constants!
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
```

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(32, 32);
  dim3 gridDim((4000 + 32 - 1) / 32, (5000 + 32 - 1) / 32); // Ceil division
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

Naive kernel launch

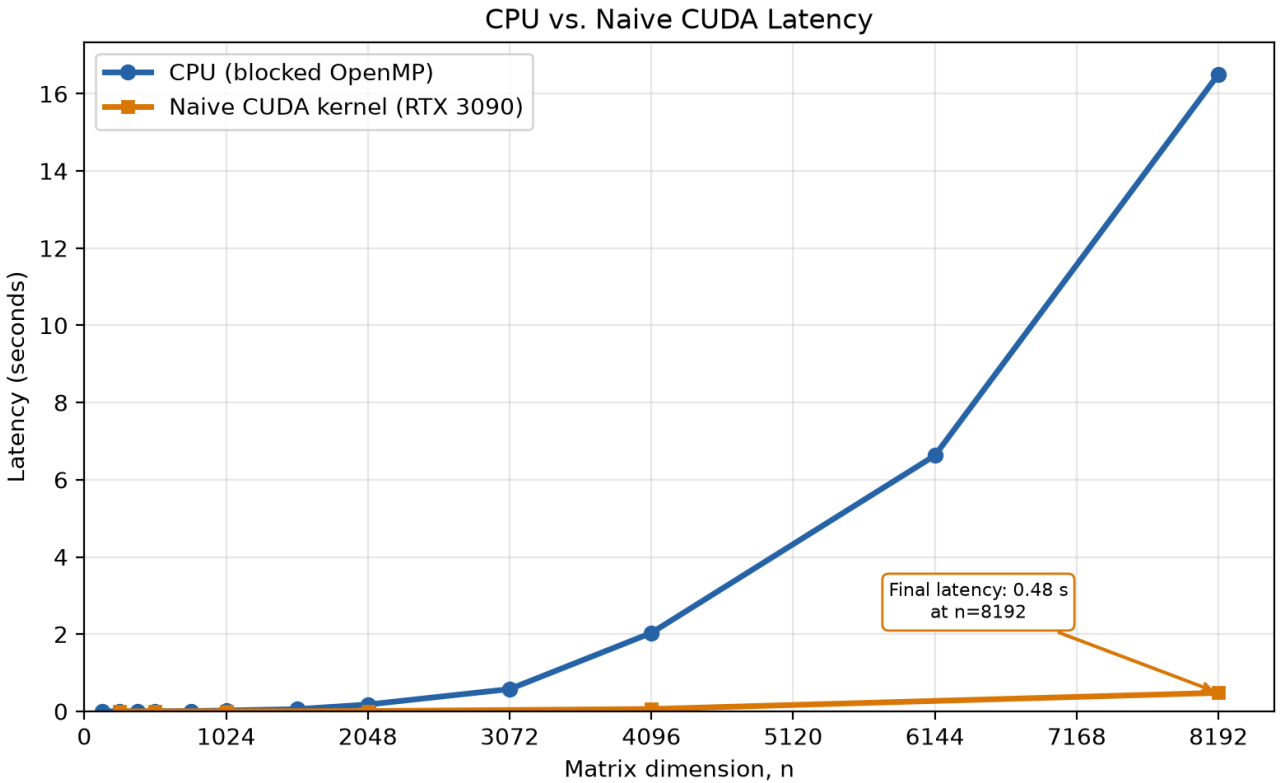
Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(32, 32);
  dim3 gridDim(126, 158); // Ceil division
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

Naive CUDA is already faster than the CPU

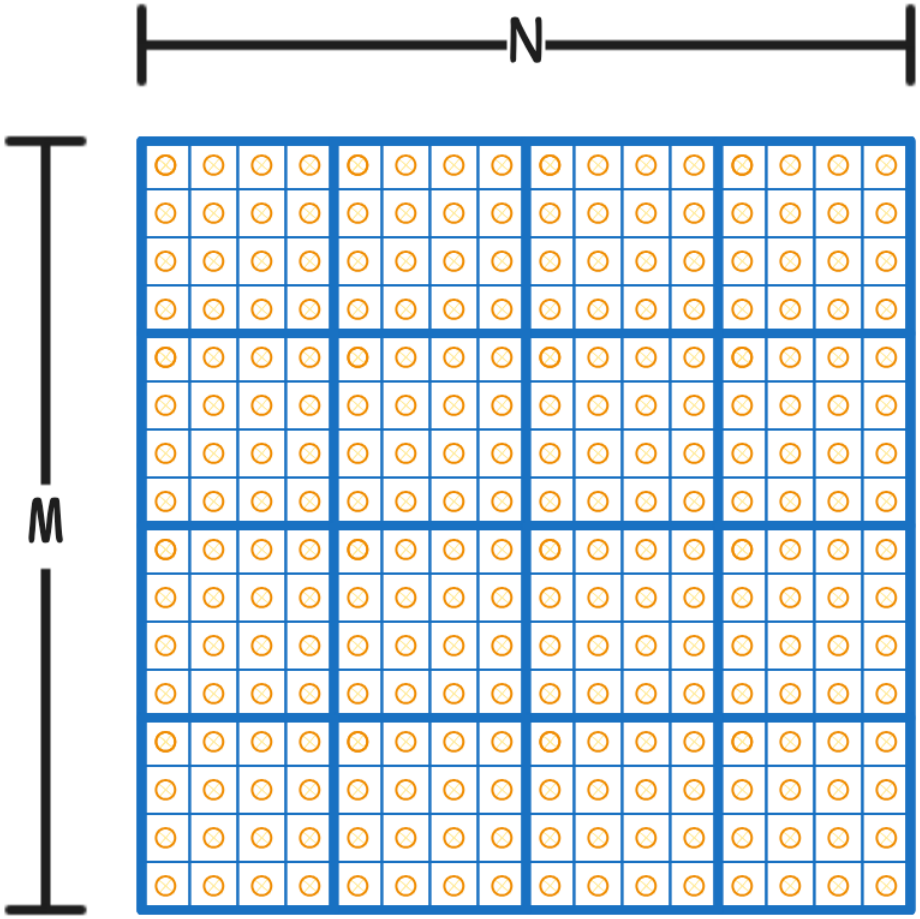


Section Overview

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

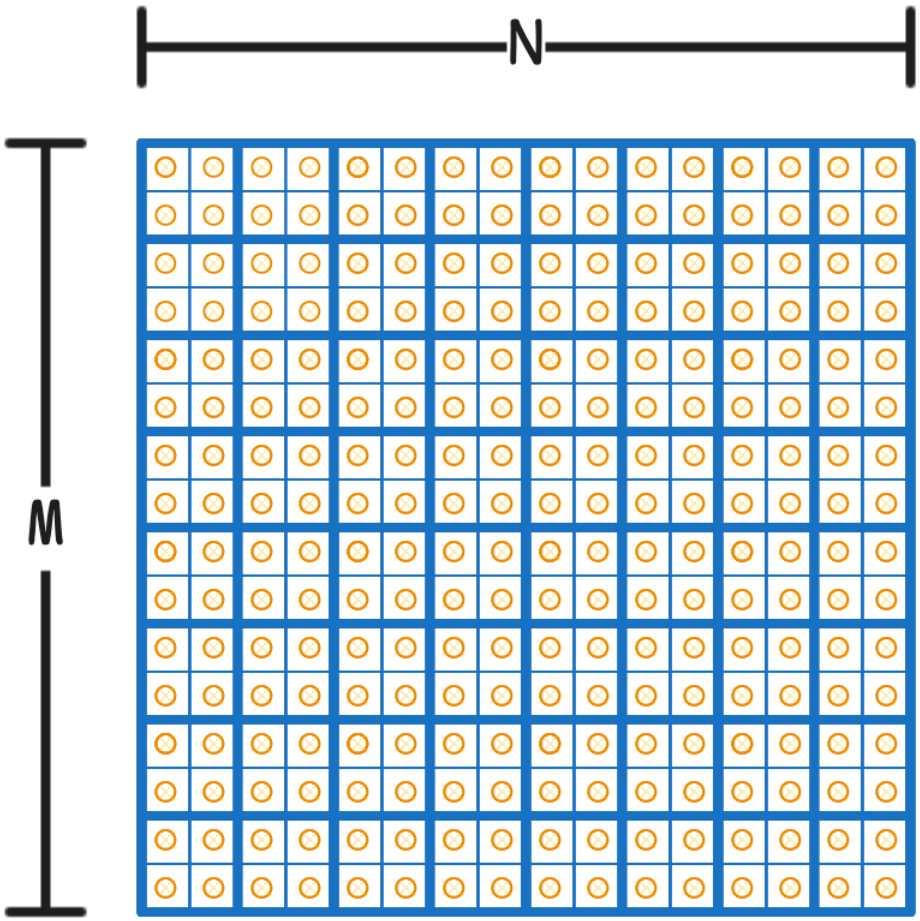
Let's do some experimenting

This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?



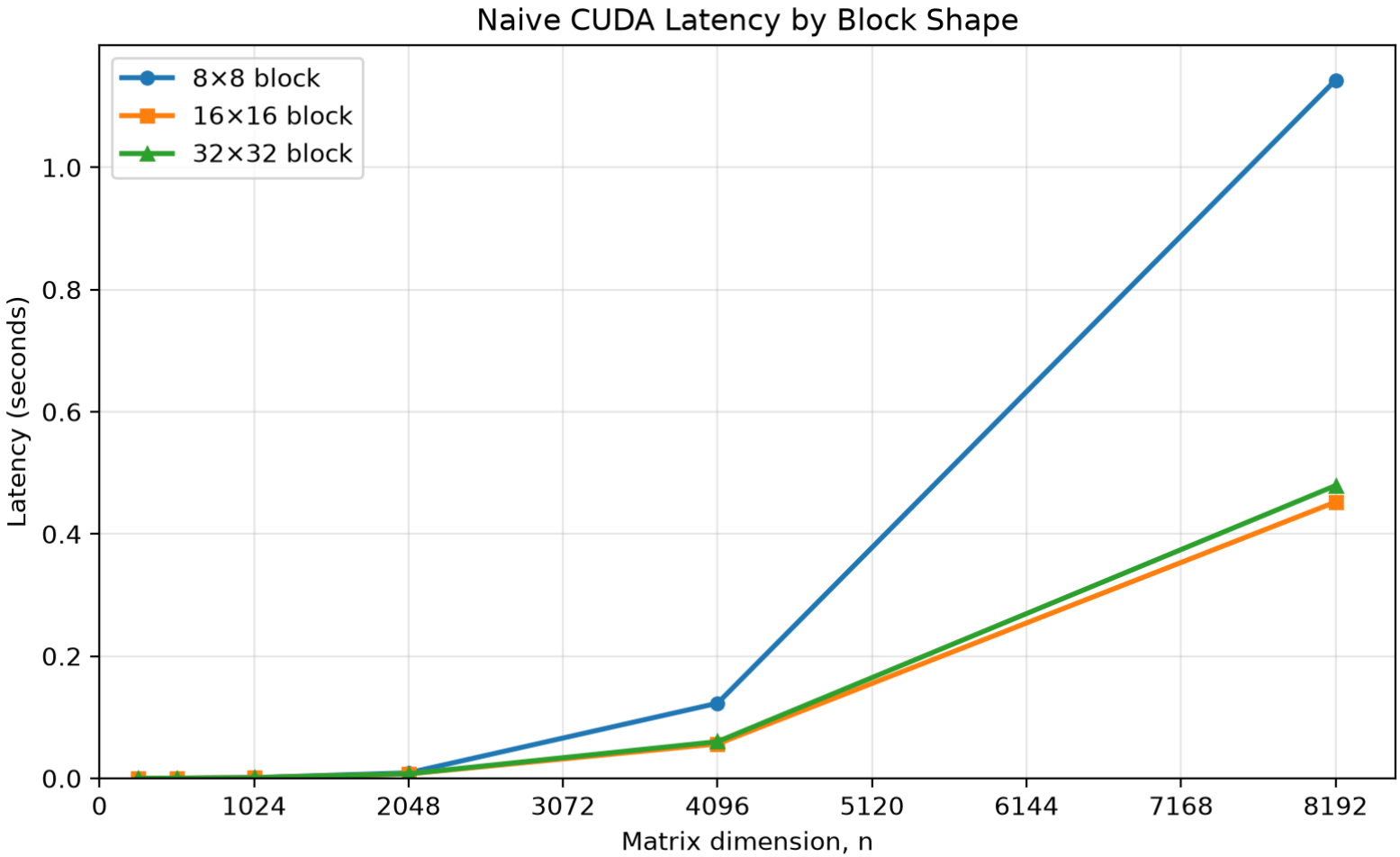
Let's do some experimenting

This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?



Let's do some experimenting

This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?



Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's.
The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's.
The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

Block shape	**L1 Cache Hit Rate**
8×8	83.51%
16×16	87.13%
32×32	94.98%

RTX 3090, $M = N = K = 4096$

Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's. The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

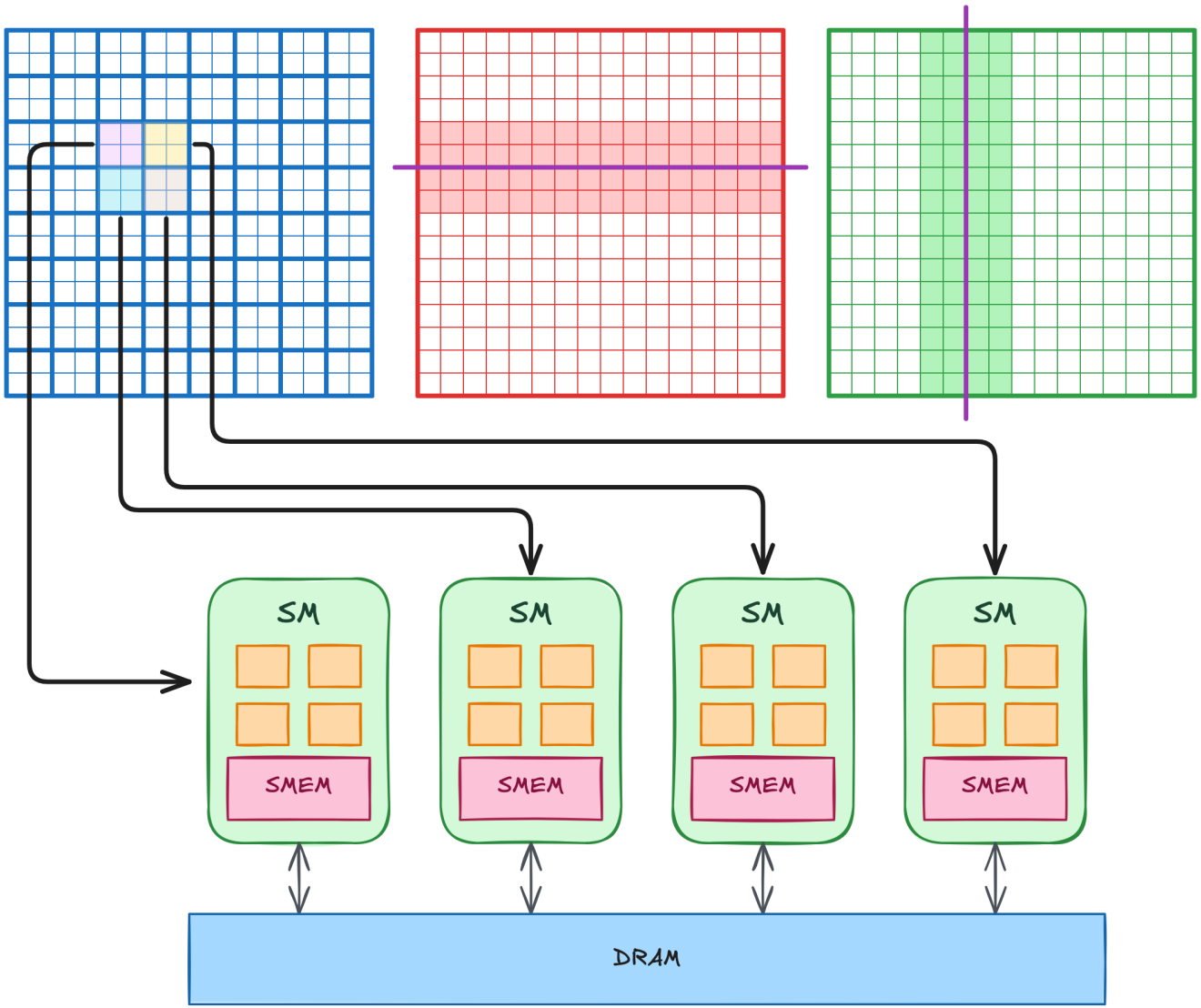
Block shape	**L1 Cache Hit Rate**
8×8	83.51%
16×16	87.13%
32×32	94.98%

RTX 3090, $M = N = K = 4096$

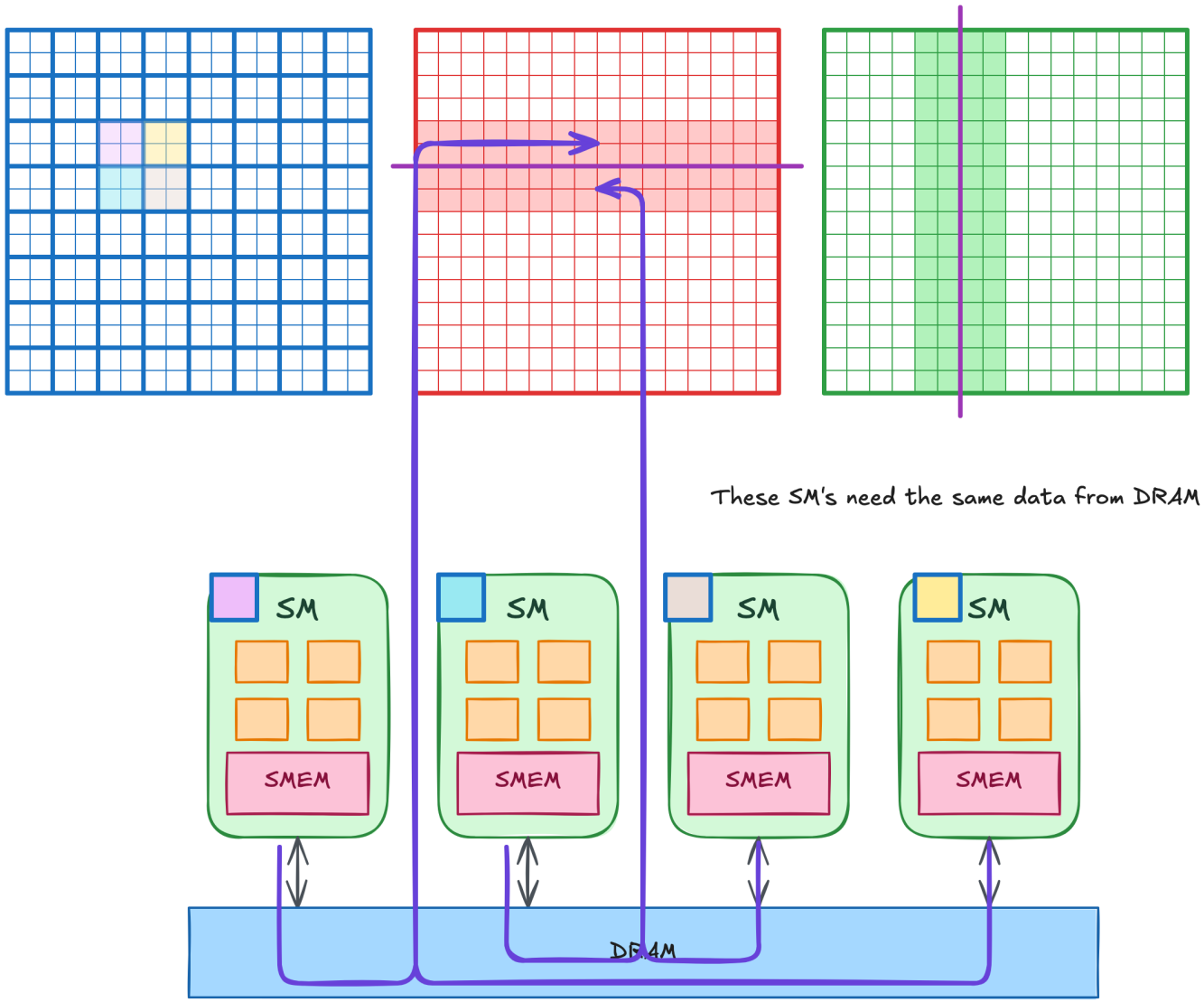
Intuition

Nearby elements of C require **nearby elements in** A **and** B. Blocks are assigned to SM's. Therefore, larger blocks mean that larger patches of C are assigned the to same SM's, increasing the chances of L1 cache hits.

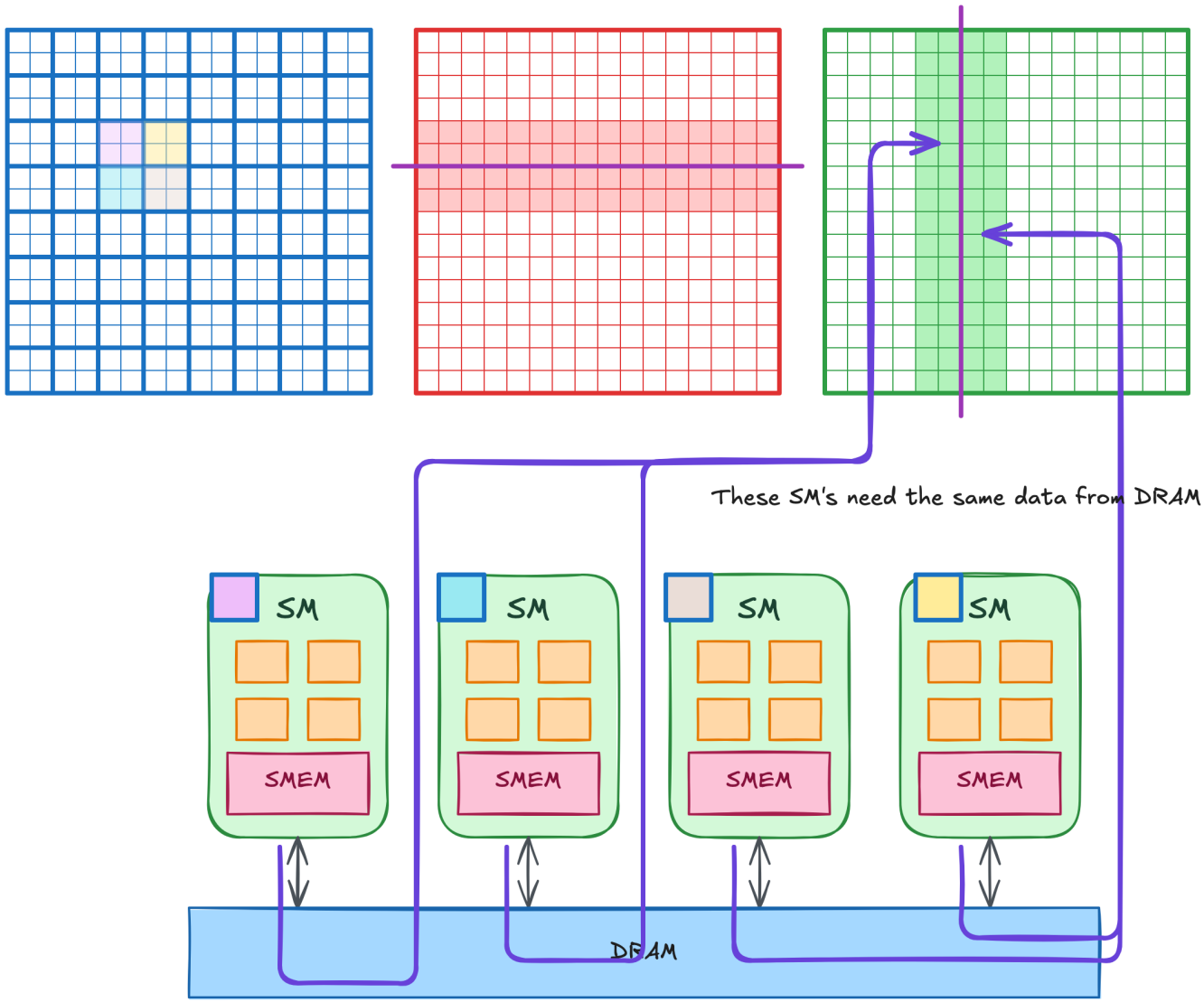
Why Larger Blocks Improve Cache Reuse



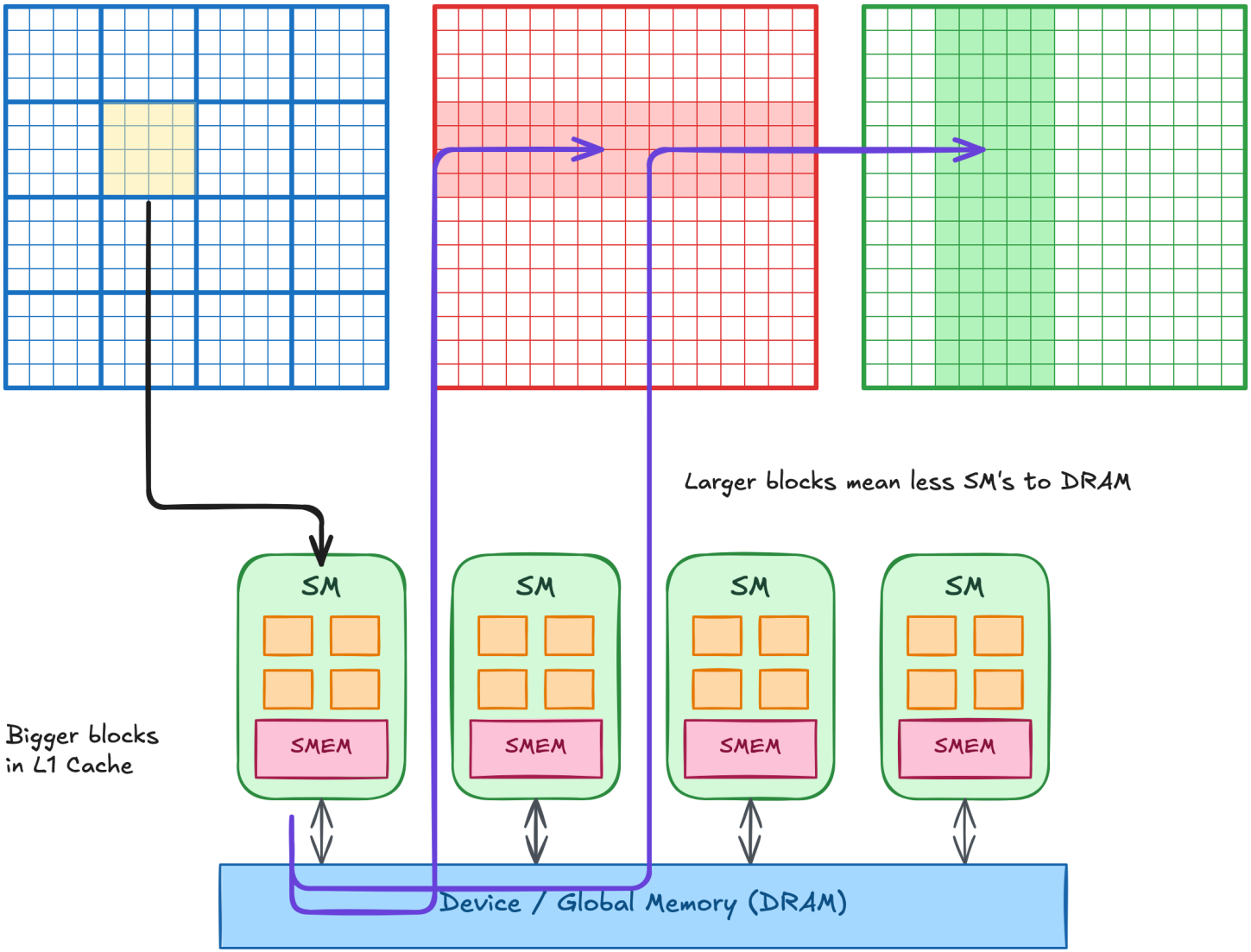
Why Larger Blocks Improve Cache Reuse



Why Larger Blocks Improve Cache Reuse



Why Larger Blocks Improve Cache Reuse



Shared Memory (SMEM) Caching

Programmable L1 Cache (SMEM)

Remember how our launch config required us to group threads as blocks? That is because **blocks** are scheduled to SM's as a unit, so **threads in the same block have access to the SM's L1 cache/SMEM.**

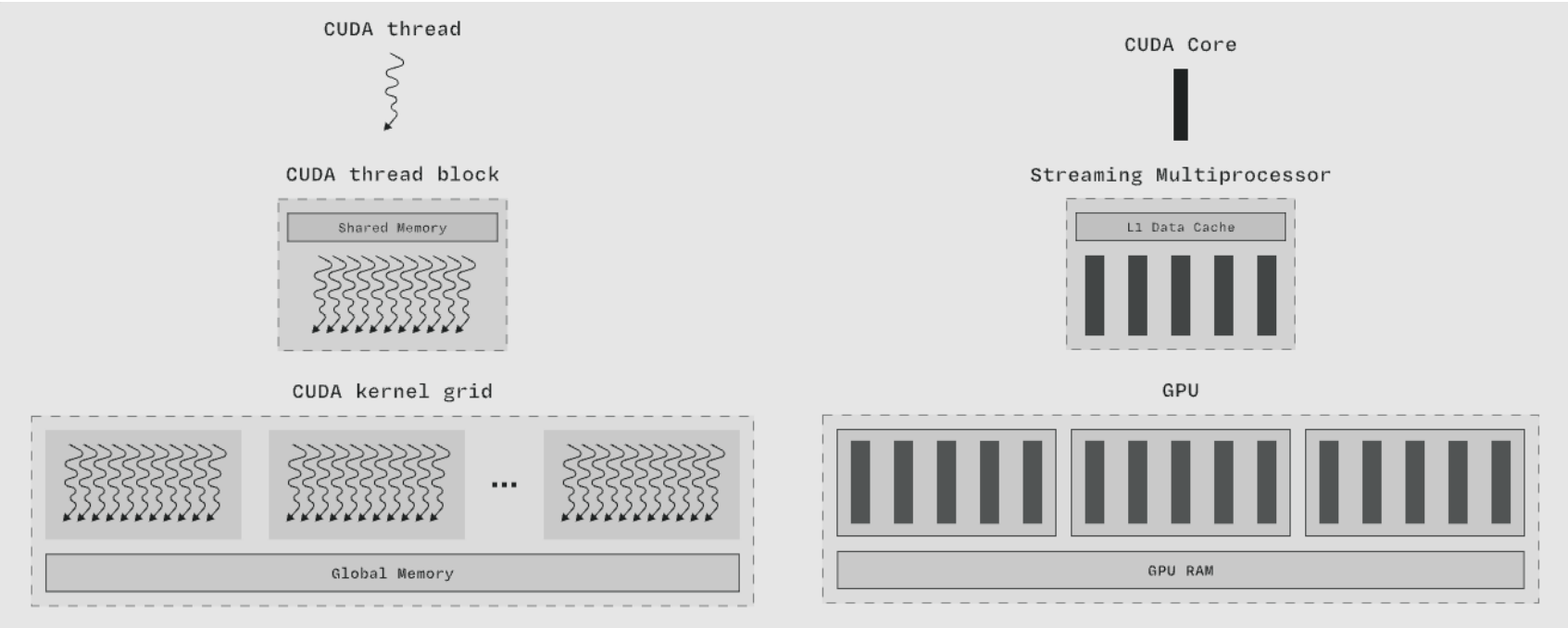


Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

Tiling: Explicit Cache use

Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.

Basic tiled SGEMM: step by step

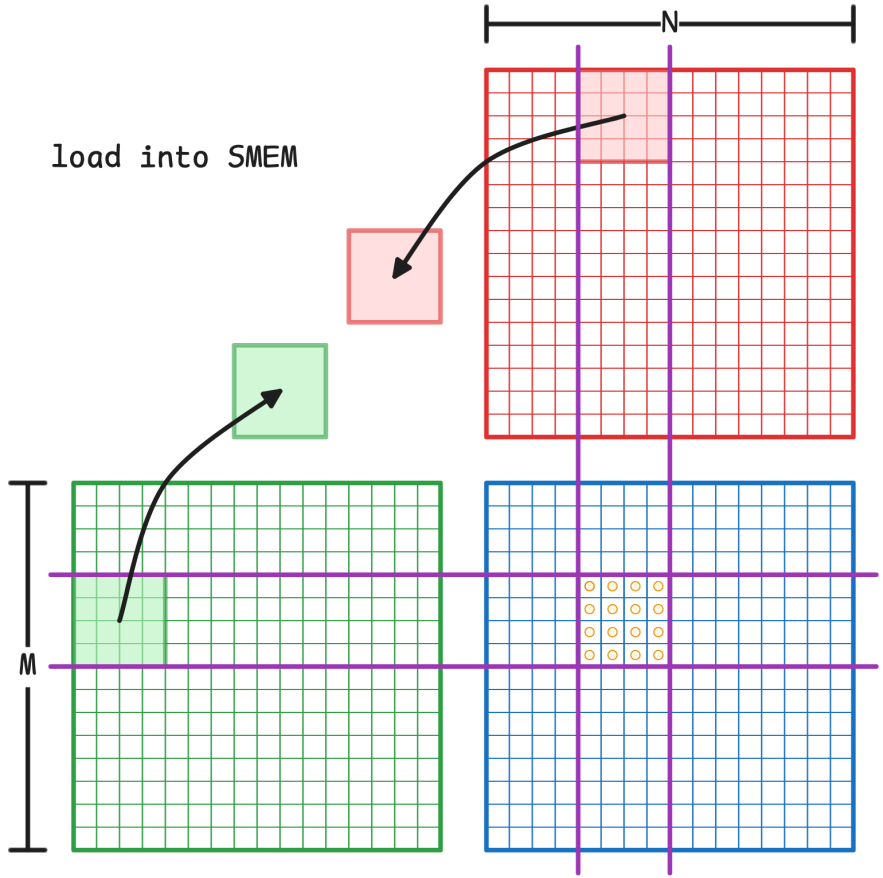
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

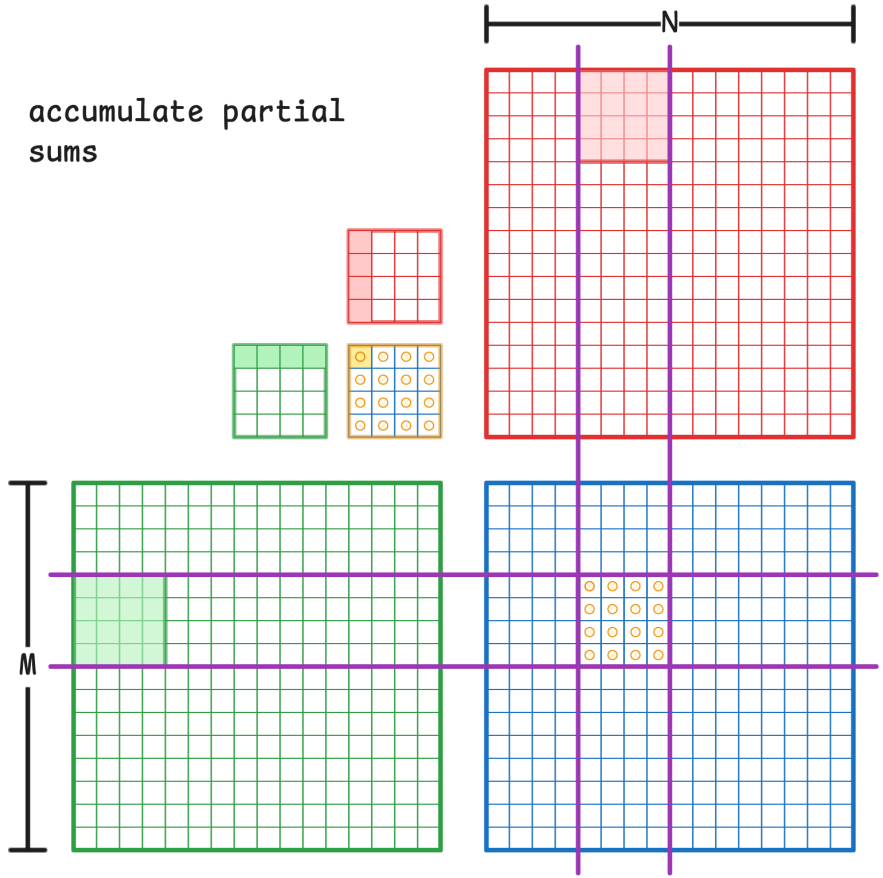
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

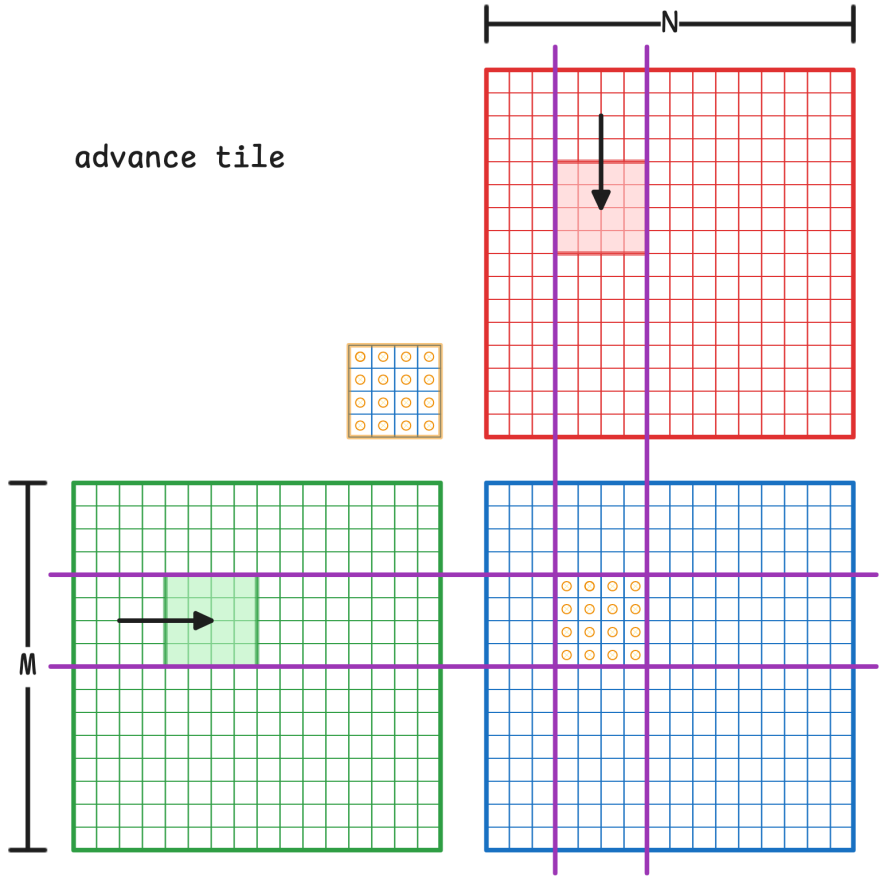
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

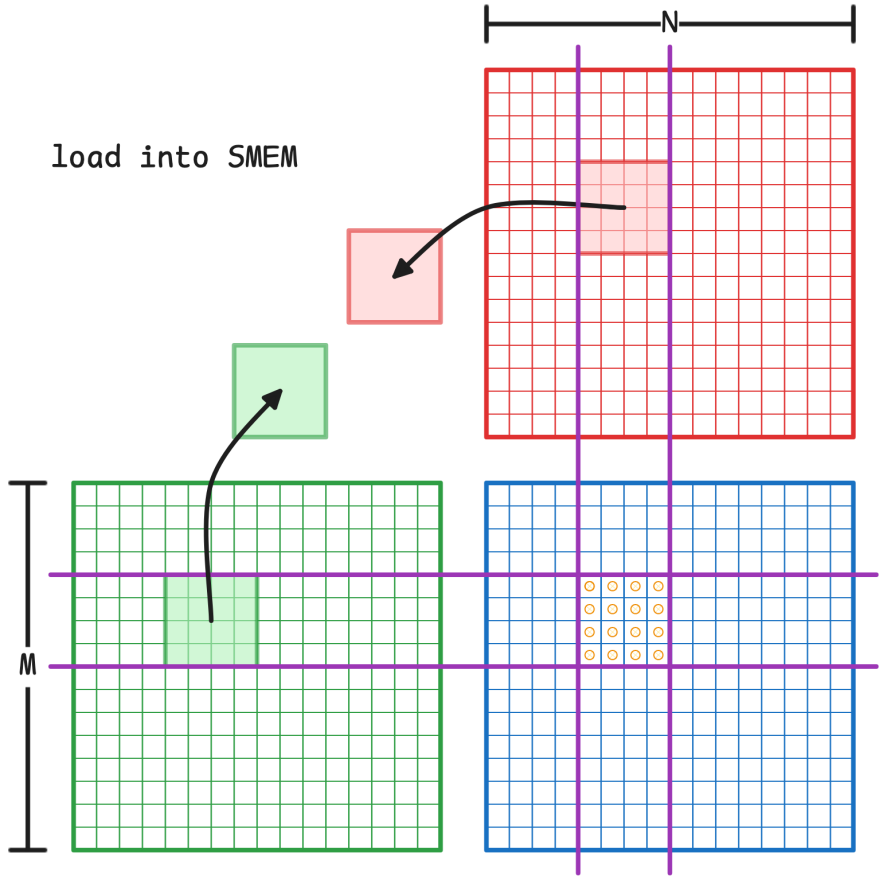
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

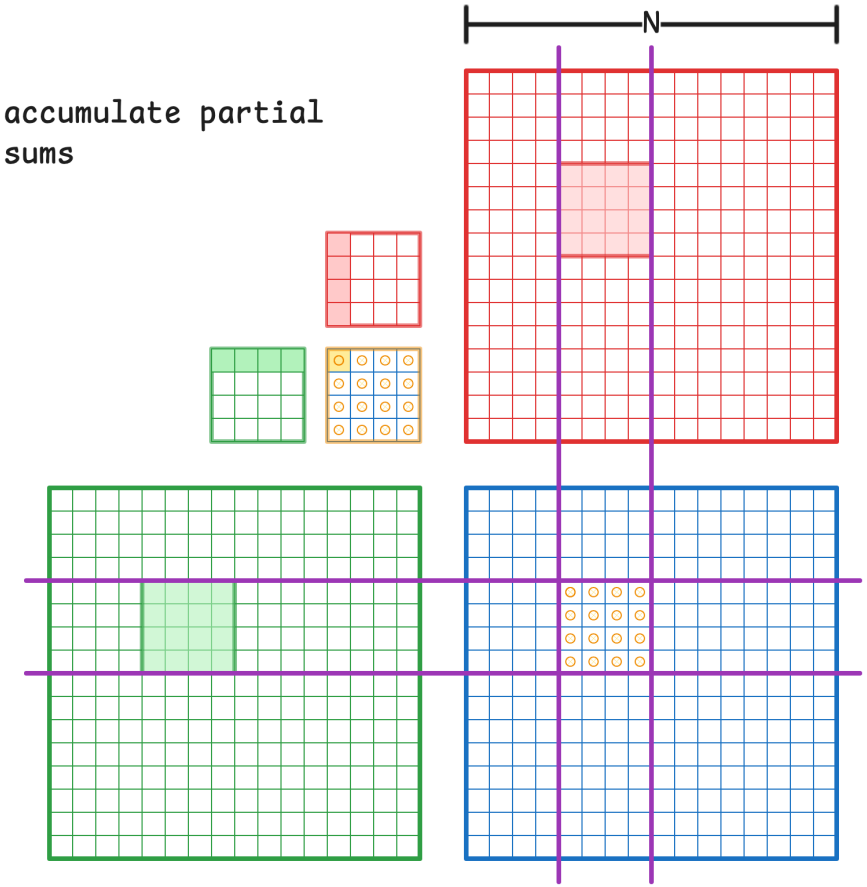
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

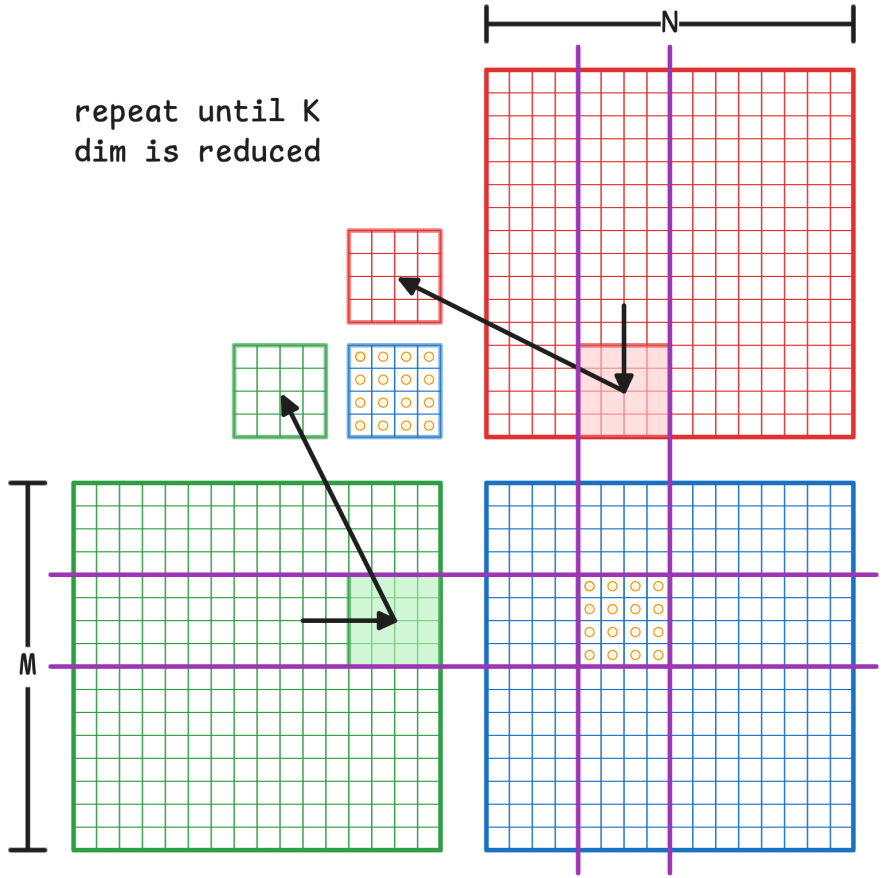
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```

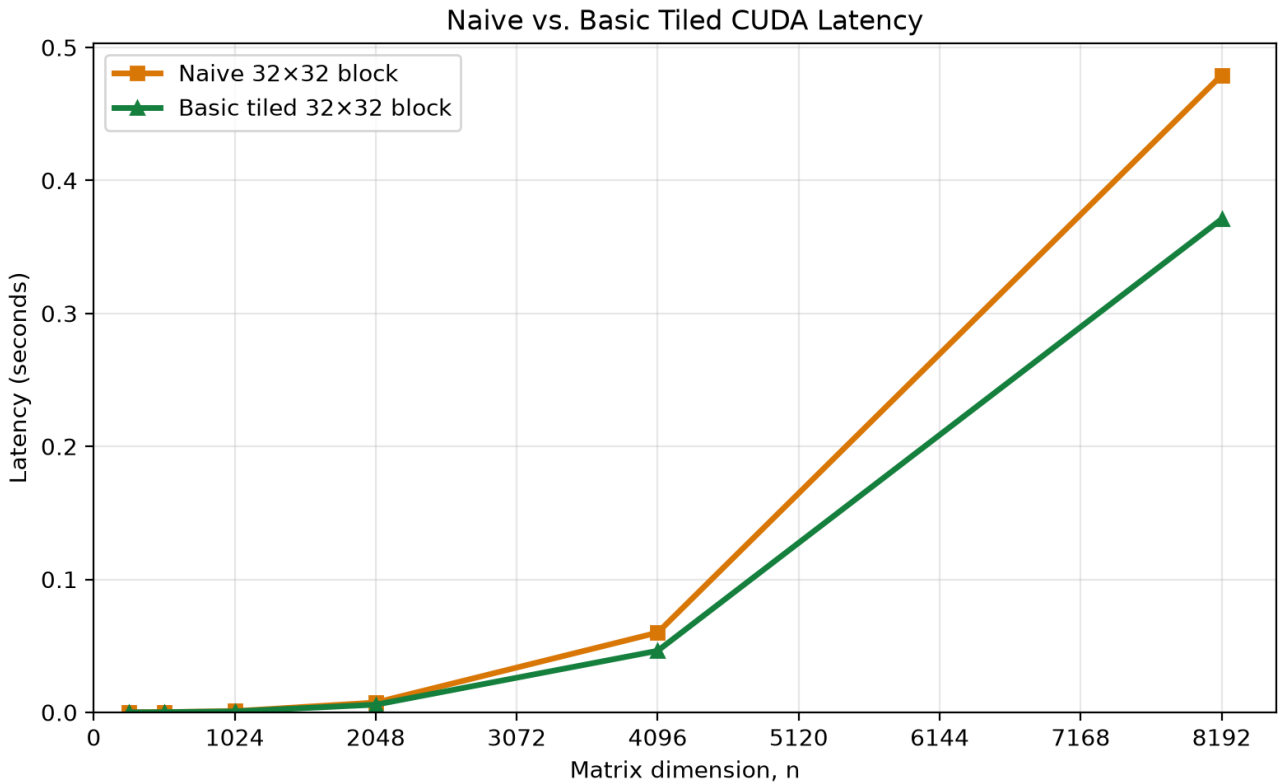


Basic tiled SGEMM visualization

- Watch each block load small regions of A and B together.
- The loaded tiles are reused to update an entire tile of C.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

Tiling performance



At $n, m, k = 8192$, tiled 32×32 latency is 371 ms, a 22.6% improvement over the naive.

Section Overview

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

Driving Question

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Is there a better performance metric?

We have been measuring our kernels with **latency**, but is there some other metric that better describes "performance"?

Driving Question

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Is there a better performance metric?

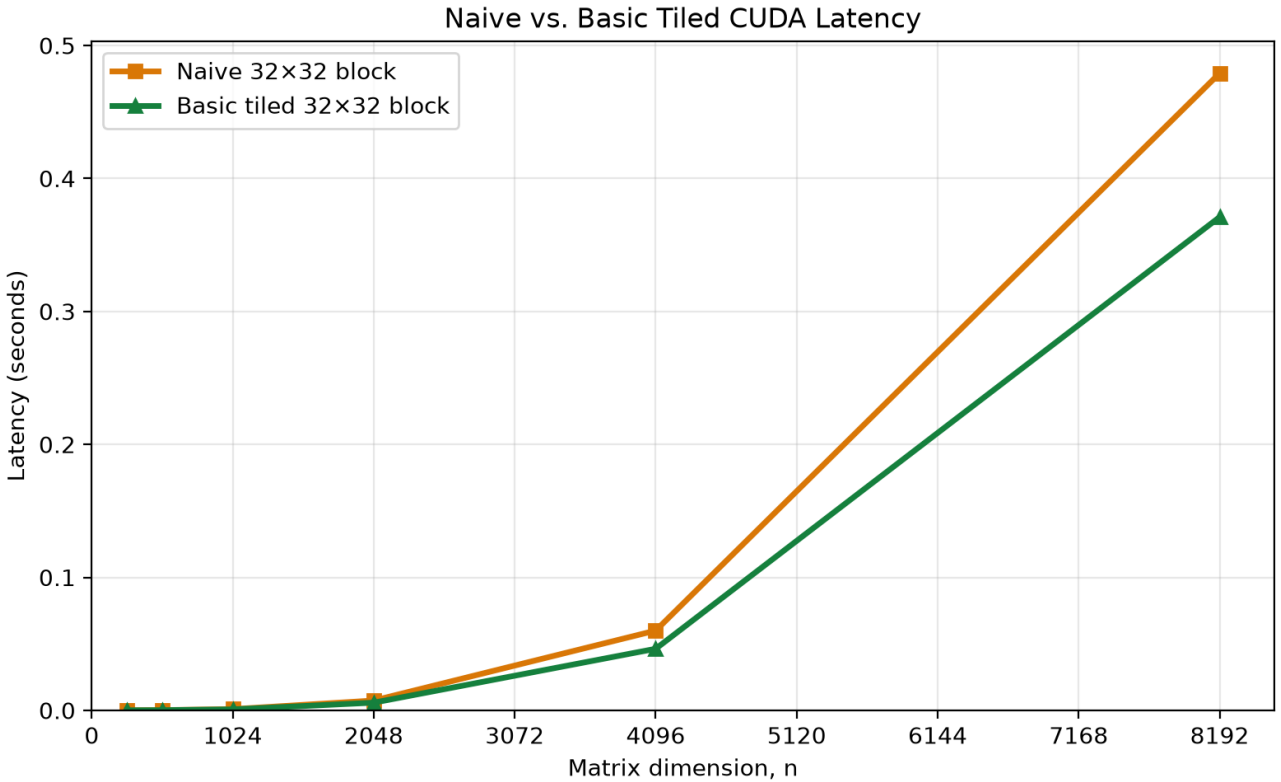
We have been measuring our kernels with **latency**, but is there some other metric that better describes "performance"?

How fast *can* we be?

Given our hardware constraints, how fast (theoretically) can we do our SGEMM operations?

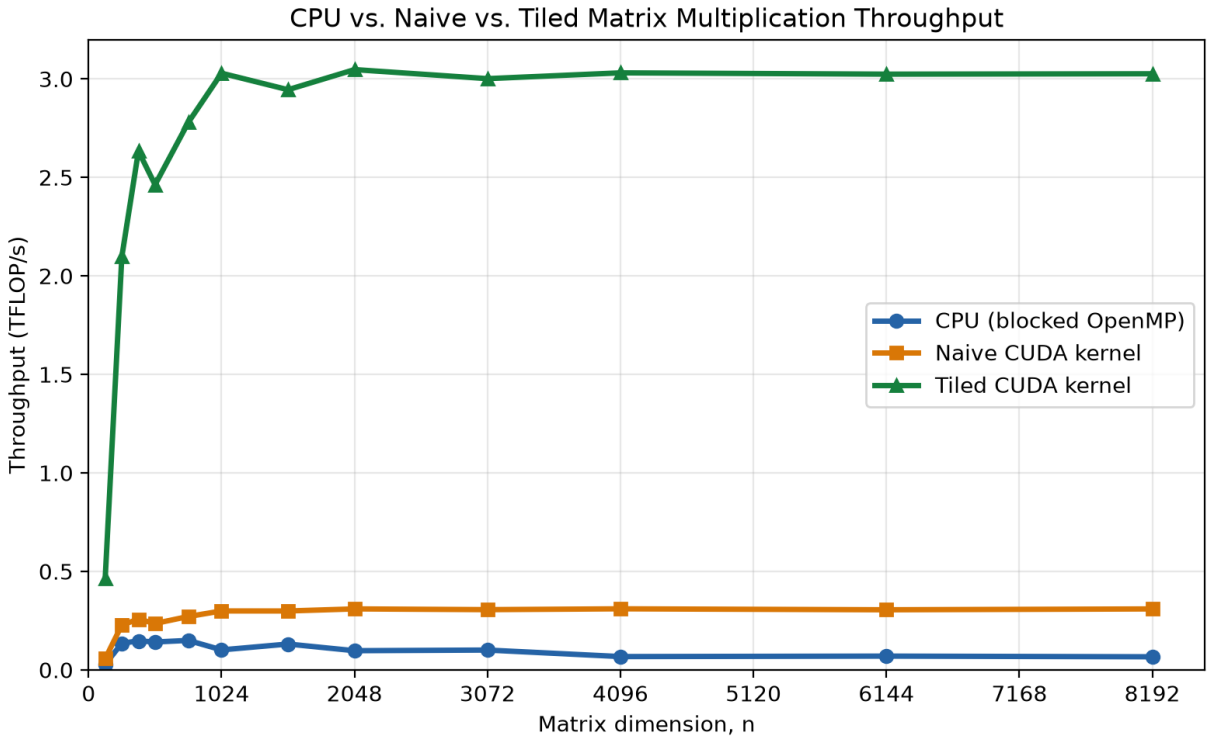
Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.



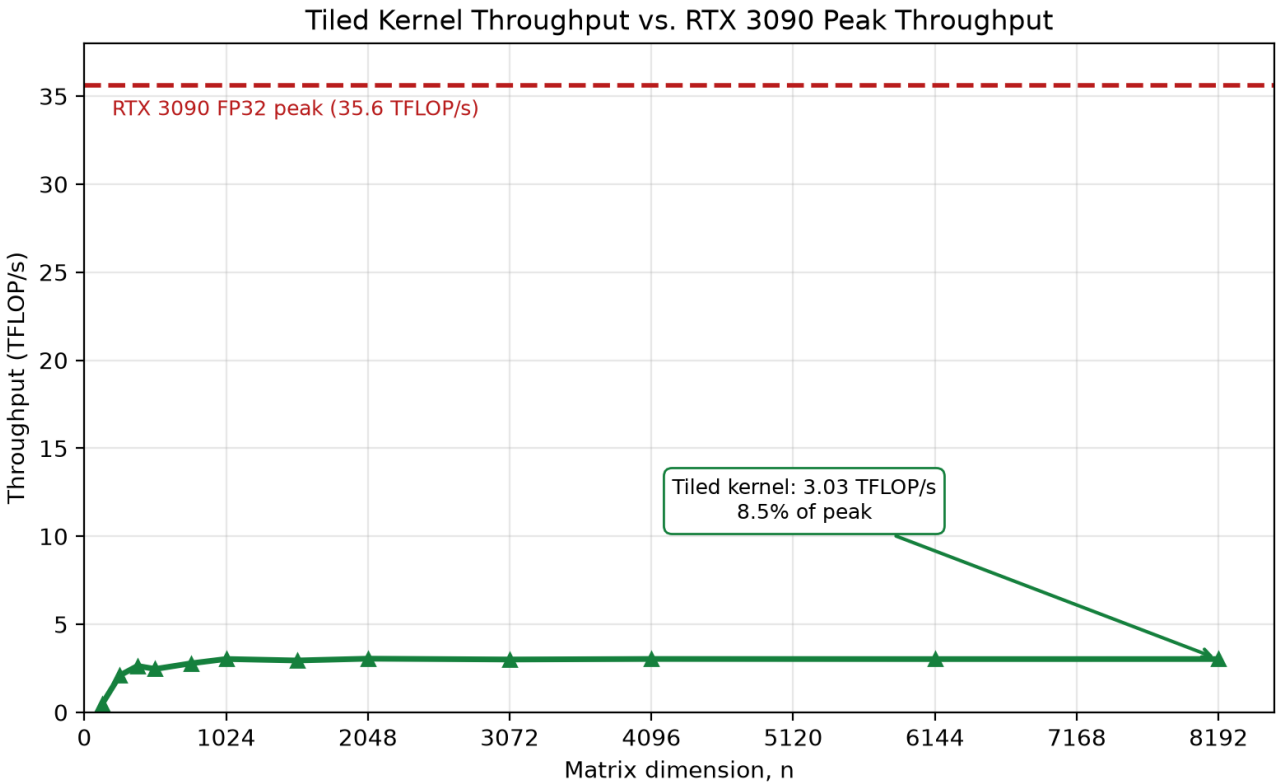
Why latency alone isn't enough

- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.



Why latency alone isn't enough

- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.



Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.
- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.
- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.

Why throughput

Throughput tells us

1. A general performance metric, no matter the problem size
2. How well we are utilizing hardware

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time.

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time.

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example (Measuring Tiled Kernel)

We observed 360ms or 0.36s of latency for $n = 8192$, where the total FLOPs required is $2(8192)^3 \approx 1.10$ TFLOPs. Thus, we have an achieved throughput of

$$\frac{1.10\text{TFLOPs}}{0.36\text{s}} \approx 3.05\,\text{TFLOP/s}.$$

Roughly 8.5% of peak throughput $35.6\,\text{TFLOP/s}$.

Characterizing our kernels data use

1. Throughput characterizes how well our kernel utilizes the **compute hardware**.

Characterizing our kernels data use

1. Throughput characterizes how well our kernel utilizes the **compute hardware**.
2. One factor that impacts final throughput is how efficiently our kernels use data.

Characterizing our kernels data use

1. Throughput characterizes how well our kernel utilizes the **compute hardware**.
2. One factor that impacts final throughput is how efficiently our kernels use data.

How can we quantify how "well" our kernel uses data?

We observed that we can improve the performance of our SGEMM kernel with tiling, an optimization that improves *data use*.

Characterizing our kernels data use

1. Throughput characterizes how well our kernel utilizes the **compute hardware**.
2. One factor that impacts final throughput is how efficiently our kernels use data.

How can we quantify how "well" our kernel uses data?

We observed that we can improve the performance of our SGEMM kernel with tiling, an optimization that improves *data use*. **Can we quantify this data usage?**

Arithmetic Intensity: Characterizing our kernels

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Arithmetic Intensity: Characterizing our kernels

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Example (Naive kernel)

For $n, m, k = 8192$, ignoring hardware L1 caching, we load (from DRAM)

$$n \times m \times 2k \times 4 \text{ bytes} \approx 4398 \text{ GB}.$$

We do 1.10 TFLOPs of work, so we have an AI of

$$\text{AI} = \frac{1.10 \text{ TFLOP}}{4.398 \text{ TB}} \approx 0.25 \text{ FLOP/byte}.$$

Arithmetic Intensity: Characterizing our kernels

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Example (Tiled kernel)

For $n, k, m = 8192$, in the worst case, our tiled kernel loads (from DRAM)

$$\frac{m}{32} \times \frac{n}{32} \times 2\left(\frac{k}{32}\right) \times 32 \times 32 \times 4 \text{ bytes} = 137.4 \text{ GB}$$

With a total work count of 1.10 TFLOP, its arithmetic intensity is

$$\text{AI} = \frac{1.10 \text{ TFLOP}}{137.7 \text{ GB}} \approx 8.0 \text{ FLOP/byte}.$$

The Memory Roof

Memory Roof

Given the DRAM bandwidth and a kernel's AI, we say the **memory roof** of the kernel is

$$\text{Memory Roof} = \text{AI} \times \text{DRAM bandwidth}.$$

The Memory Roof

Memory Roof

Given the DRAM bandwidth and a kernel's AI, we say the **memory roof** of the kernel is

$$\text{Memory Roof} = \text{AI} \times \text{DRAM bandwidth}.$$

Example (Intuition)

In order for the CUDA Cores to do work, we have to first move the data from DRAM to the SM's.

The Memory Roof

Memory Roof

Given the DRAM bandwidth and a kernel's AI, we say the **memory roof** of the kernel is

$$\text{Memory Roof} = \text{AI} \times \text{DRAM bandwidth}.$$

Example (Intuition)

In order for the CUDA Cores to do work, we have to first move the data from DRAM to the SM's. If the kernel only does a small amount of work on the data (low AI), it will likely finish before the next batch of data arrives.

The Memory Roof

Memory Roof

Given the DRAM bandwidth and a kernel's AI, we say the **memory roof** of the kernel is

$$\text{Memory Roof} = \text{AI} \times \text{DRAM bandwidth}.$$

Example (Intuition)

In order for the CUDA Cores to do work, we have to first move the data from DRAM to the SM's. If the kernel only does a small amount of work on the data (low AI), it will likely finish before the next batch of data arrives. **In this case, the final measured throughput will be constrained by the DRAM bandwidth.** We are **memory bound.**

The Memory Roof

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example (Tiled Kernel)

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.8 $\mathrm{GB/s}$ of bandwidth, our **memory roof** is

$$935.8\,\mathrm{GB/s} \times 8.0\ \mathrm{FLOP/byte} \approx 7.49\,\mathrm{TFLOP/s}.$$

The Memory Roof

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example (Tiled Kernel)

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.8 $\mathrm{GB/s}$ of bandwidth, our **memory roof** is

$$935.8\,\mathrm{GB/s} \times 8.0\ \mathrm{FLOP/byte} \approx 7.49\,\mathrm{TFLOP/s}.$$

This is much lower than the peak throughput of 35.6 $\mathrm{TFLOP/s}$.

The Memory Roof

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example (Tiled Kernel)

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At $935.8\,\mathrm{GB/s}$ of bandwidth, our **memory roof** is

$$935.8\,\mathrm{GB/s} \times 8.0\ \mathrm{FLOP/byte} \approx 7.49\,\mathrm{TFLOP/s}.$$

This is much lower than the peak throughput of $35.6\,\mathrm{TFLOP/s}$. **No matter how we optimize this kernel, without increasing AI, we will never achieve greater than** $7.49\,\mathrm{TFLOP/s}$.

Hypothetical Kernel

RTX 3090		
Peak throughput	35.6	TFLOP/s
DRAM bandwidth	935.8	GB/s

Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s.

Hypothetical Kernel

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s. Our memory roof is now

$$935.8\,\mathrm{GB/s} \times 16.0\ \mathrm{FLOP/byte} \approx 15.0\,\mathrm{TFLOP/s}.$$

Hypothetical Kernel

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s. Our memory roof is now

$$935.8\,\mathrm{GB/s} \times 16.0\ \mathrm{FLOP/byte} \approx 15.0\,\mathrm{TFLOP/s}.$$

Since $15.0\,\mathrm{TFLOP/s} < 35.6\,\mathrm{TFLOP/s}$, this kernel will still be memory bound.

Hypothetical Kernels

RTX 3090		
Peak throughput	35.6	TFLOP/s
DRAM bandwidth	935.8	GB/s

AI (FLOP/byte)	Memory Roof (TFLOP/s)
8	7.49
16	14.97
32	29.95
36	33.69
38	35.56
40	37.43

Hypothetical Kernels

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

AI (FLOP/byte)	Memory Roof ($\mathrm{TFLOP/s}$)
8	7.49
16	14.97
32	29.95
36	33.69
38	35.56
40	37.43

Wait! Our peak throughput is 35.6 $\mathrm{TFLOP/s}$! At AI ≥ 38, we are **compute bound**. Our data can finally keep up with the CUDA Cores!

Ridge Point

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

AI $= 38$ FLOP/byte is known as the **ridge point** *RP*, and can be found by solving:

$$\text{Memory Roof} = \text{Peak throughput}$$

$$\text{DRAM Bandwidth} \times \text{RP} = \text{Peak throughput}$$

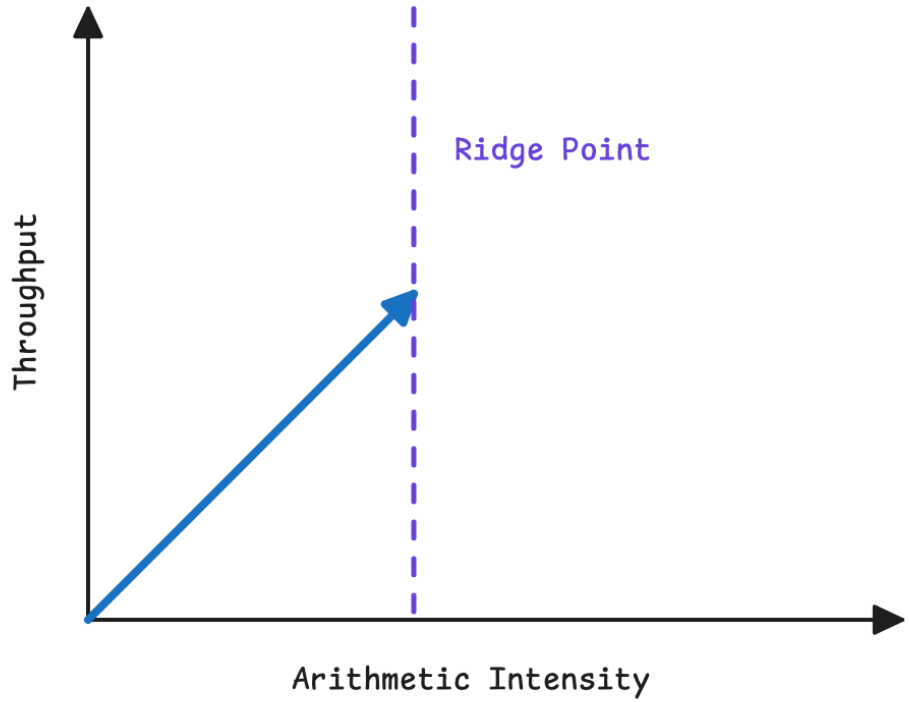
$$\text{RP} = \frac{\text{Peak throughput}}{\text{DRAM Bandwidth}}$$

$$\text{RP} = \frac{35.6\,\mathrm{TFLOP/s}}{935.8\,\mathrm{GB/s}} \approx 38.0 \text{ FLOP/s}$$

Roofline Model

Roofline Model

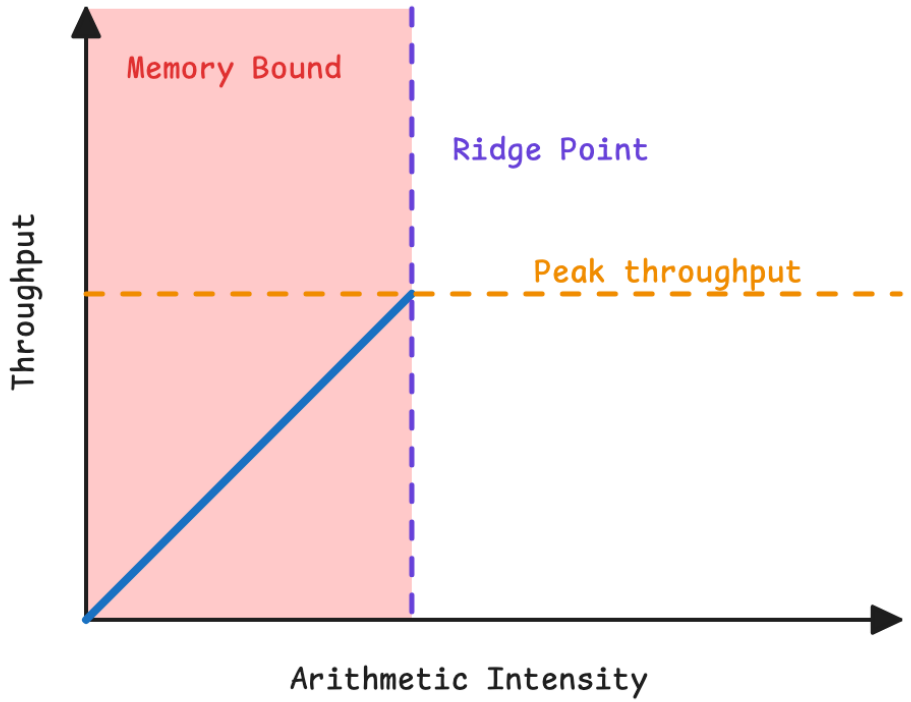
For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI.



Roofline Model

Roofline Model

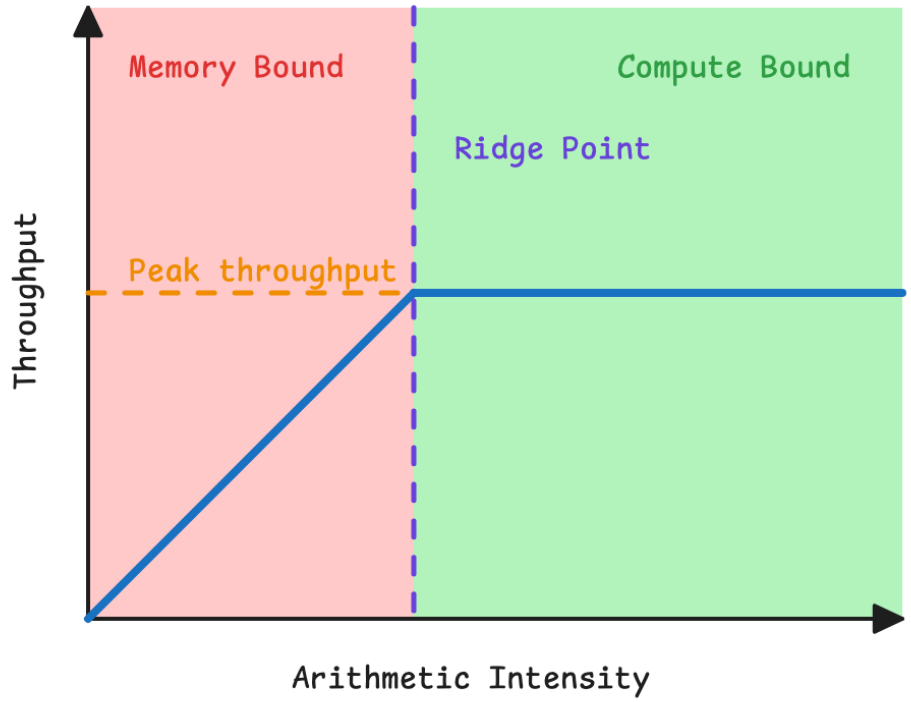
For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput.



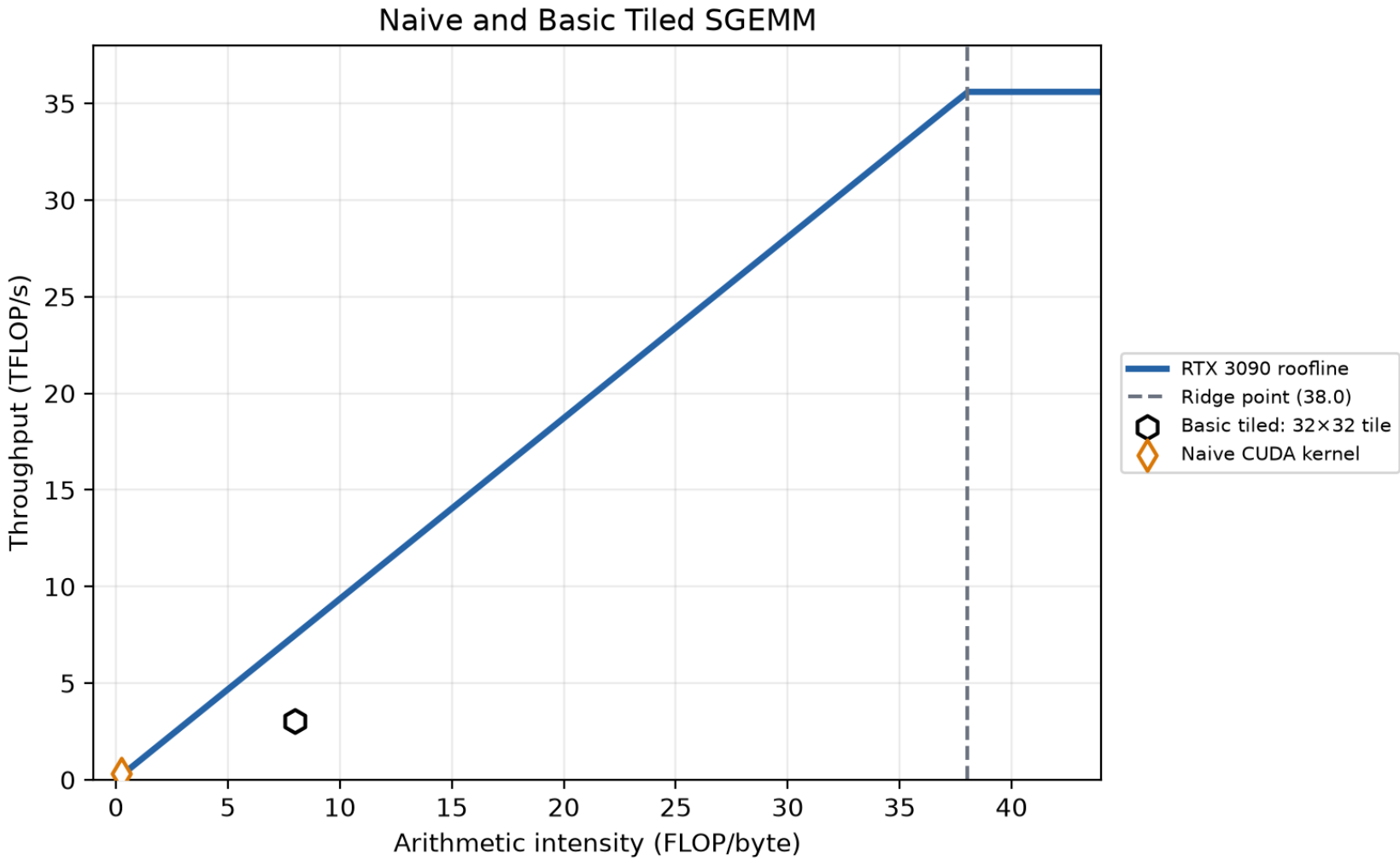
Roofline Model

Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. Past that, we are **compute bound**, increasing AI will not raise the upper bound of throughput.



Roofline Model of our RTX 3090



Since we are memory bound, increasing AI should increase performance.

Section Overview

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

Driving Question

How can we increase AI?

The roofline model tells us that increasing AI should increase throughput.

But how can we actually increase AI?

Driving Question

How can we increase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Decrease DRAM usage by utilizing SMEM more aggressively.

Driving Question

How can we increase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Decrease DRAM usage by utilizing SMEM more aggressively.
2. Loading larger tiles into SMEM.

Driving Question

How can we increase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Decrease DRAM usage by utilizing SMEM more aggressively.
2. Loading larger tiles into SMEM.
3. Reusing values in SMEM to compute multiple values of C.

Generalized Tiling: Loading larger tiles.

Our tiled kernel **hardcoded** a square 32×32 block size. Each thread in the block load one value into SMEM. We can parameterize the tiling with BM, BN, BK.

Basic tiling: one thread, one element from each tile

```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads(); // Compute partial sum using As, Bs ...
```

Direct mapping

The thread coordinates directly select the shared-memory cells:

$$(y_t, x_t) \longrightarrow \begin{cases} A_s[y_t][x_t] \\ B_s[y_t][x_t] \end{cases}$$

Limitation

This only works because the block and both shared-memory tiles are all $BK \times BK$.

General tiling: threads cooperatively load each tile

```
const int tid = yt * BN + xt;
const int numThreads = BM * BN;
__shared__ float As[BM * BK];
__shared__ float Bs[BK * BN];

for (int k = 0; k < K; k += BK) {
  for (int id = tid; id < BM * BK; id += numThreads) {
    const int yid = id / BK, xid = id % BK;
    const int A_tile_y = yb + yid;
    const int A_tile_x = k + xid;

    if (A_tile_y < M && A_tile_x < K) {
      As[id] = A[A_tile_y * K + A_tile_x];
    } else {
      As[id] = 0.0;
    }
  }

  for (int id = tid; id < BK * BN; id += numThreads) {
    const int yid = id / BK, xid = id % BK;
    const int B_tile_y = k + yid;
    const int B_tile_x = xb + xid;

    if (B_tile_y < K && B_tile_x < N) {
      Bs[id] = B[B_tile_y * N + B_tile_x];
    } else {
      Bs[id] = 0.0;
    }
  }
  __syncthreads(); // Compute partial sum using As, Bs ...
```

Thread-strided loading

Flatten each tile. Thread *tid* loads

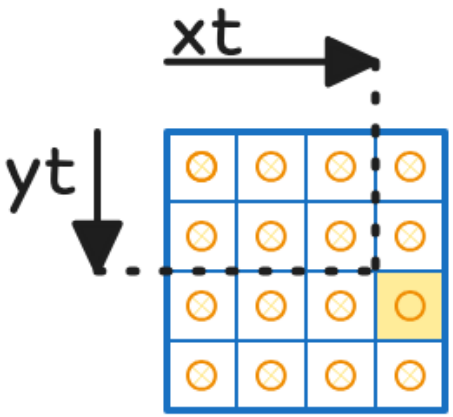
$$id = tid + jT$$

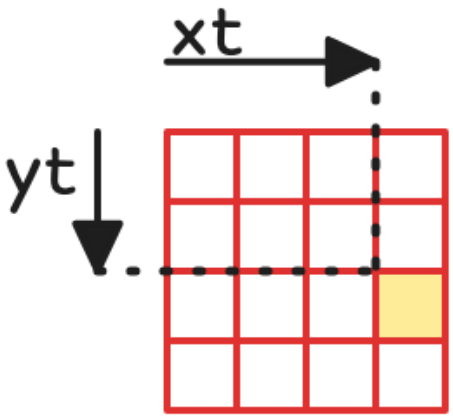
for $j = 0, 1, \ldots,$ where $T = BM \cdot BN$ is the thread count.

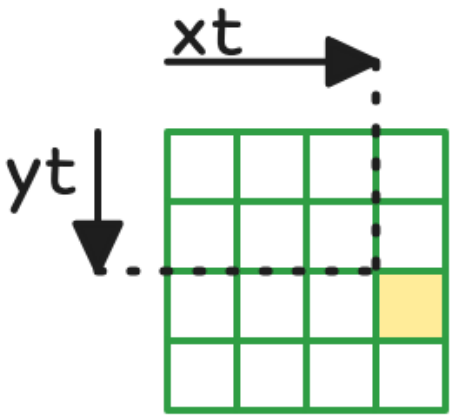
Key difference

A thread may load zero, one, or several values. Loading no longer assumes $BM = BN = BK$.

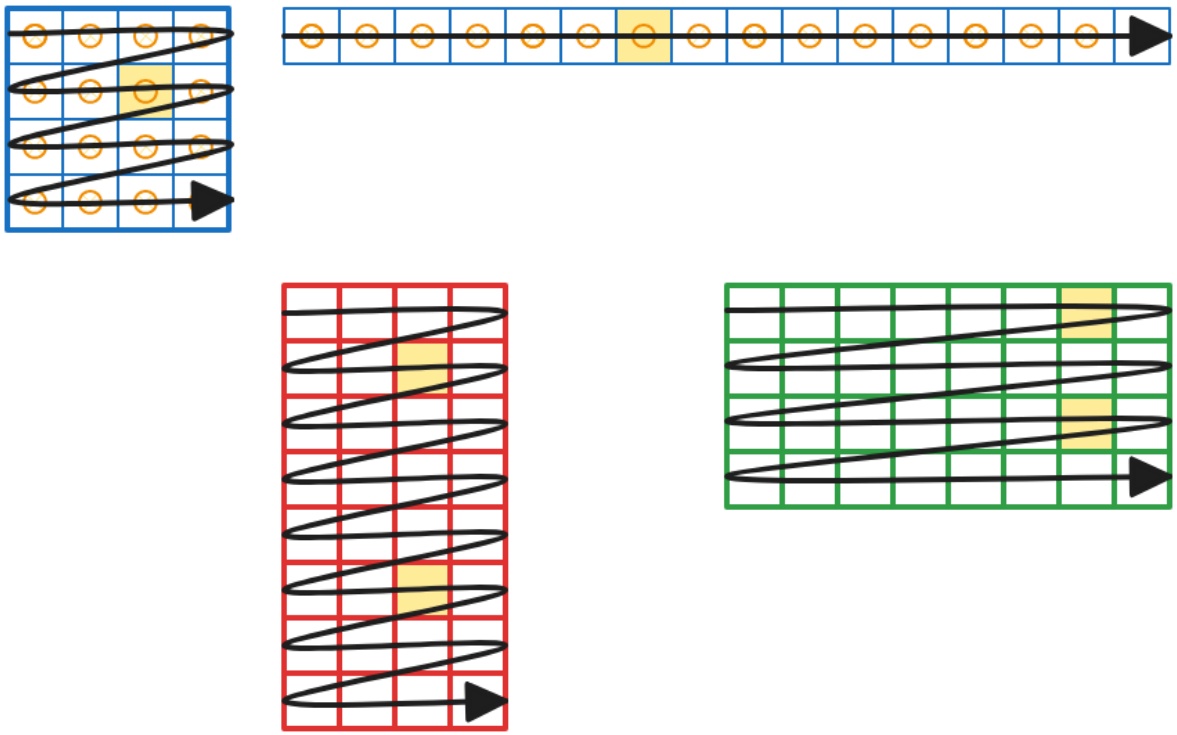
Visualization





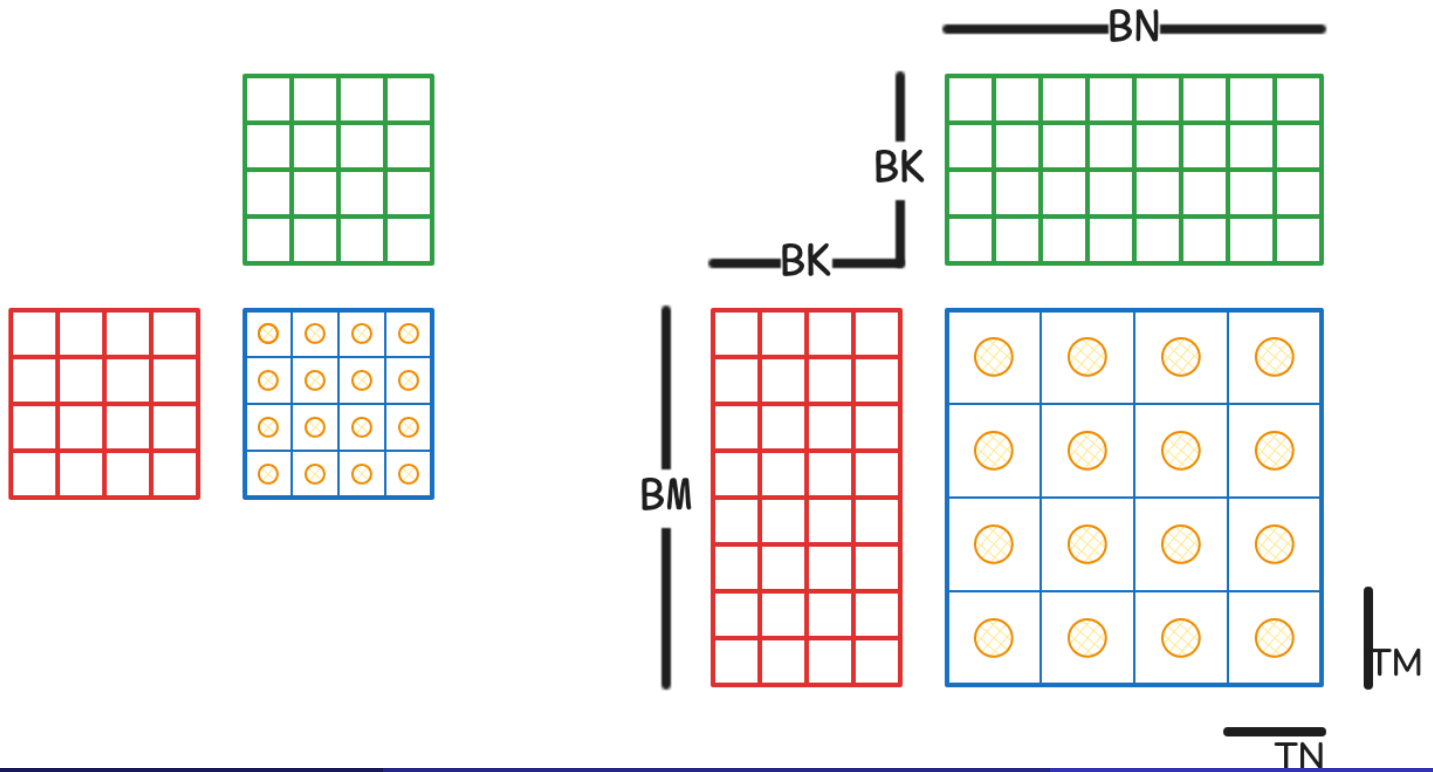


Visualization

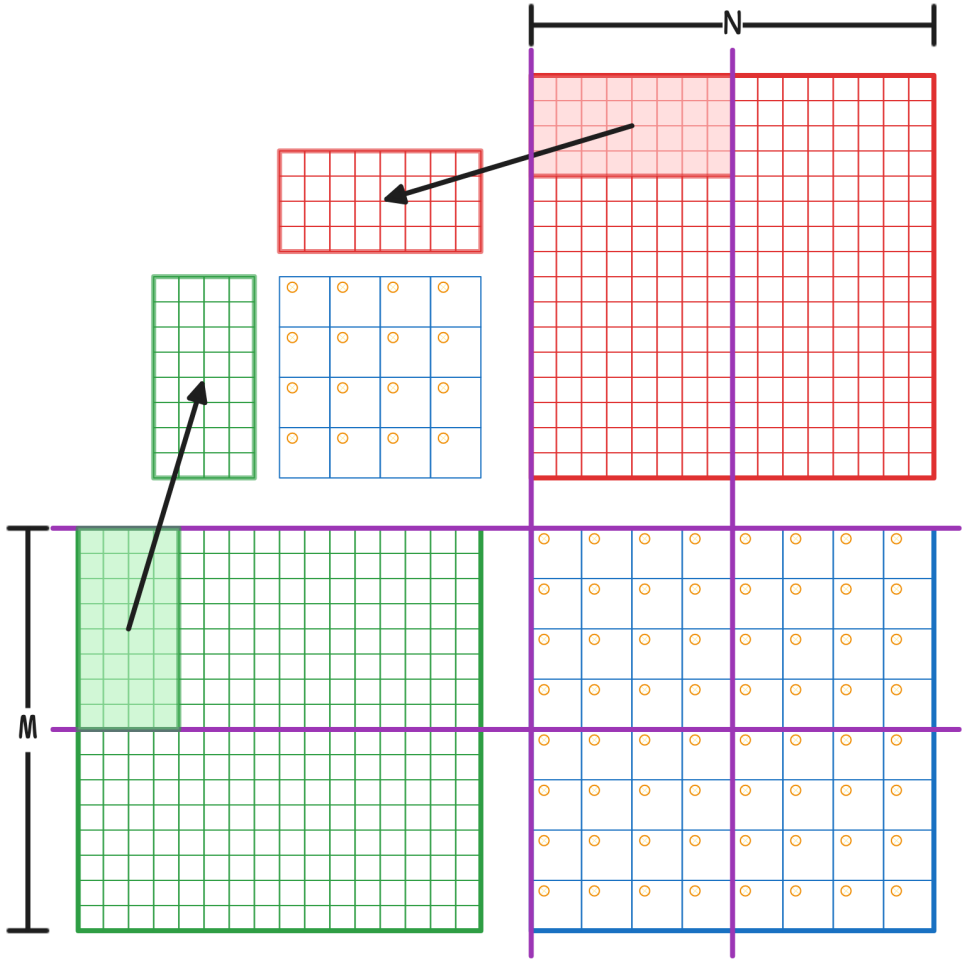


Thread coarsening: Increasing AI

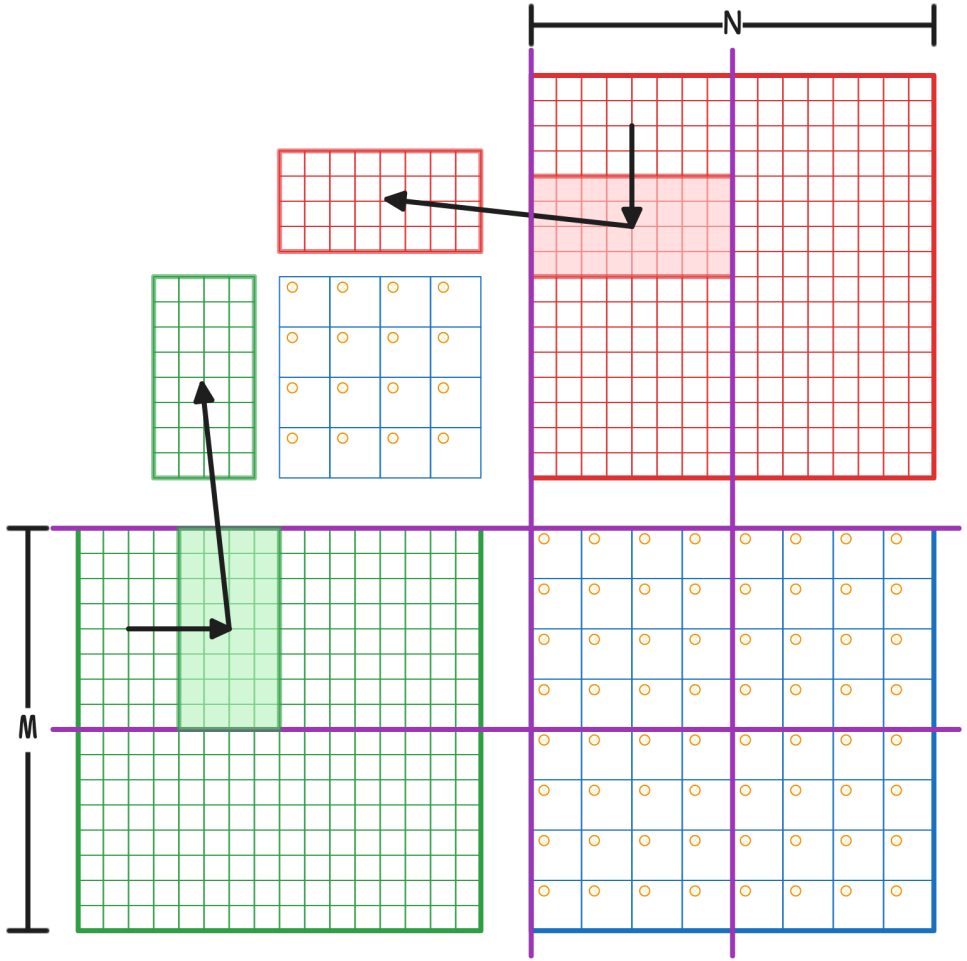
- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded A value can update several columns; one loaded B value can update several rows.



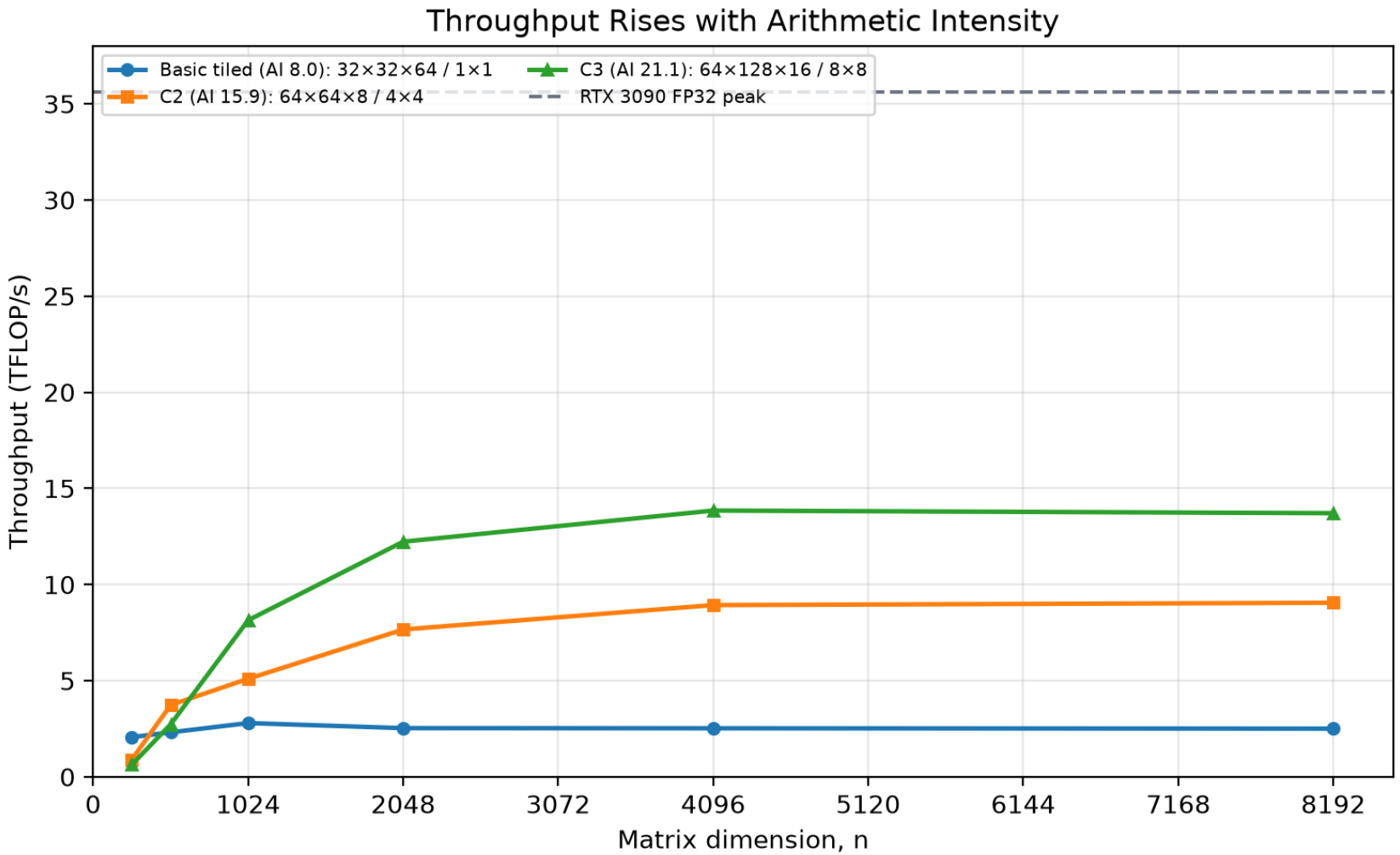
Coarsened SGEMM visualization



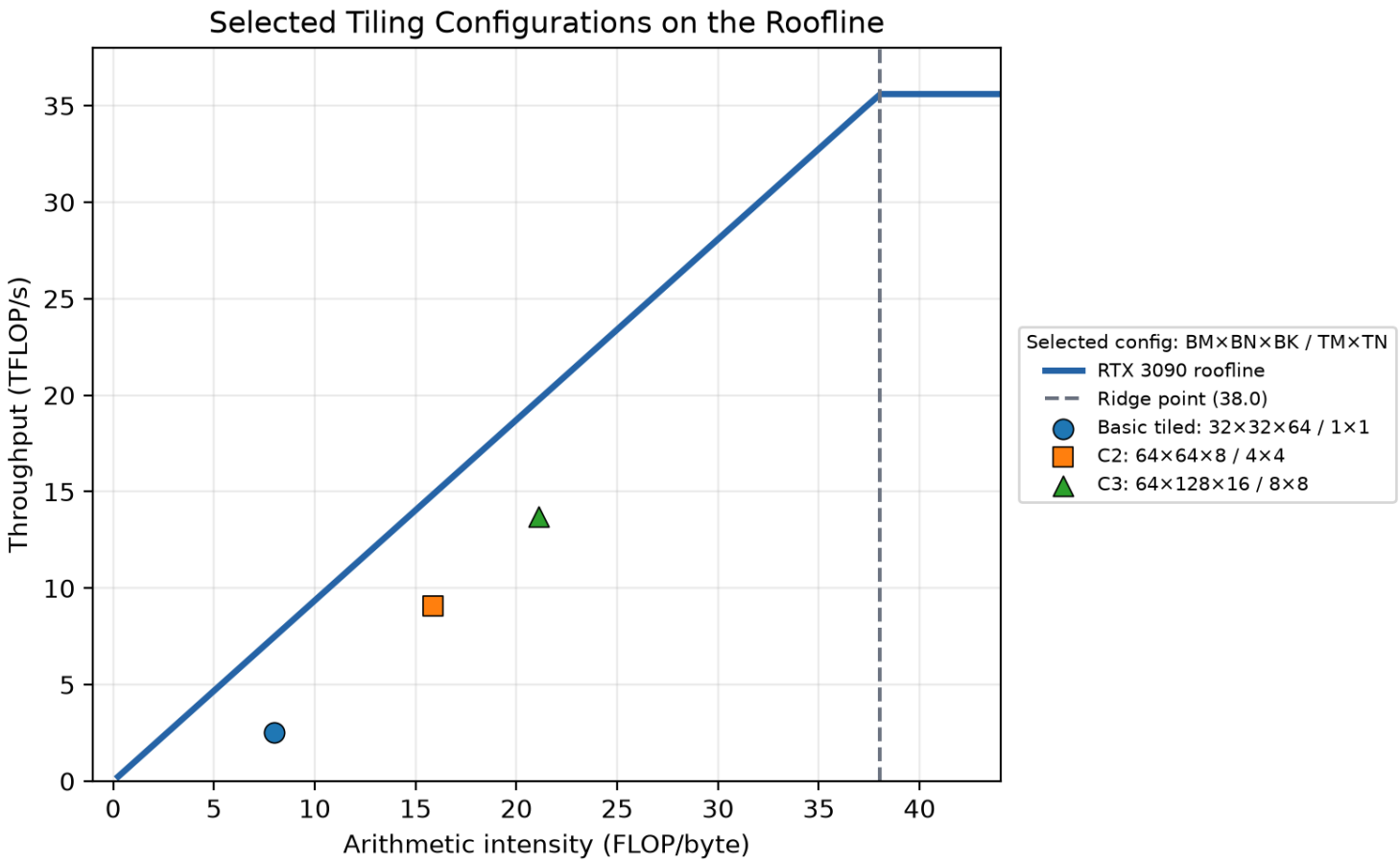
Coarsened SGEMM visualization



Higher-AI tiles tend to raise throughput

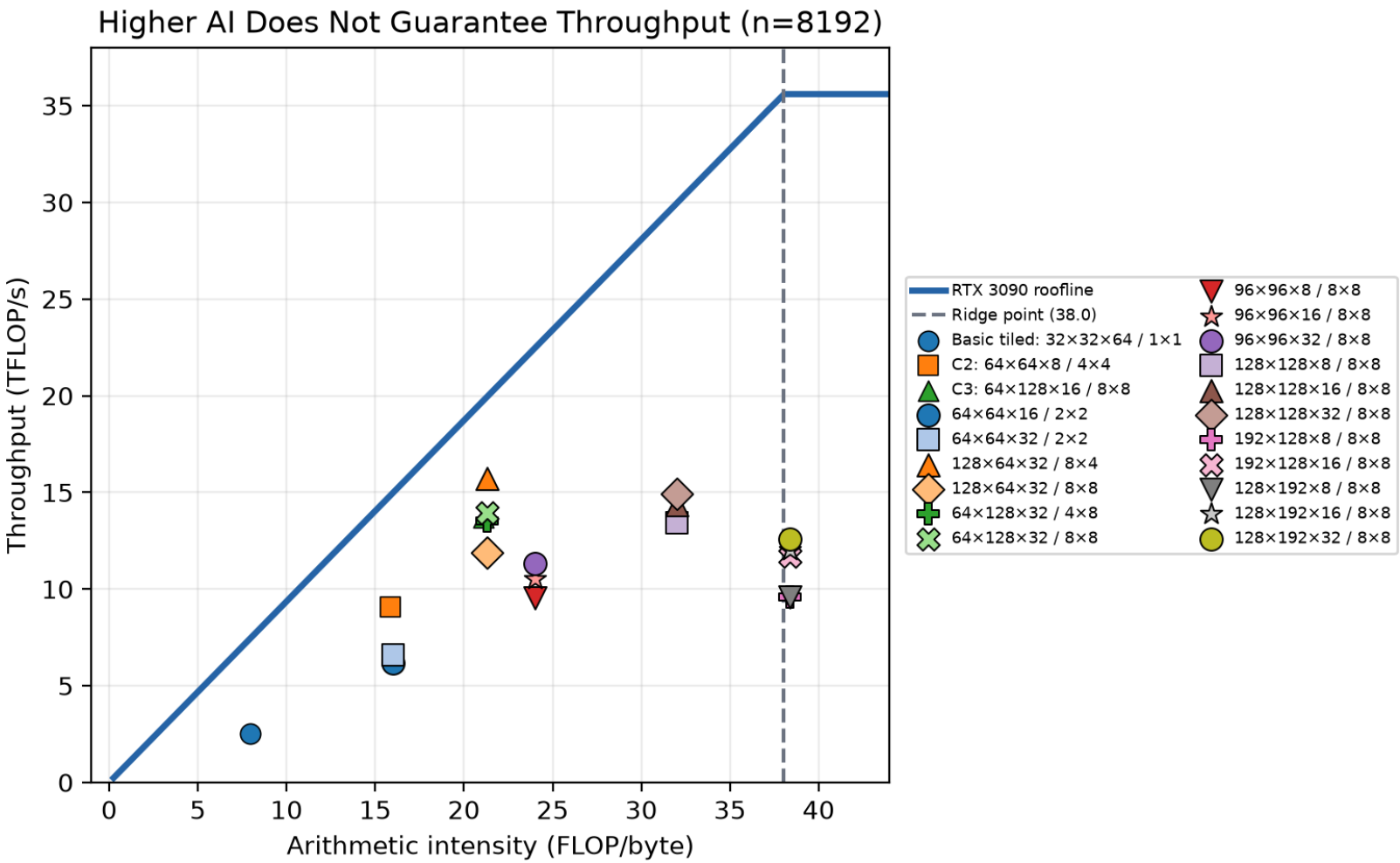


Tiling configurations on the roofline



Reminder that the Roofline Model only tells us the upper bound!

Higher AI does not guarantee higher throughput



The roofline is an upper bound, not a performance prediction.

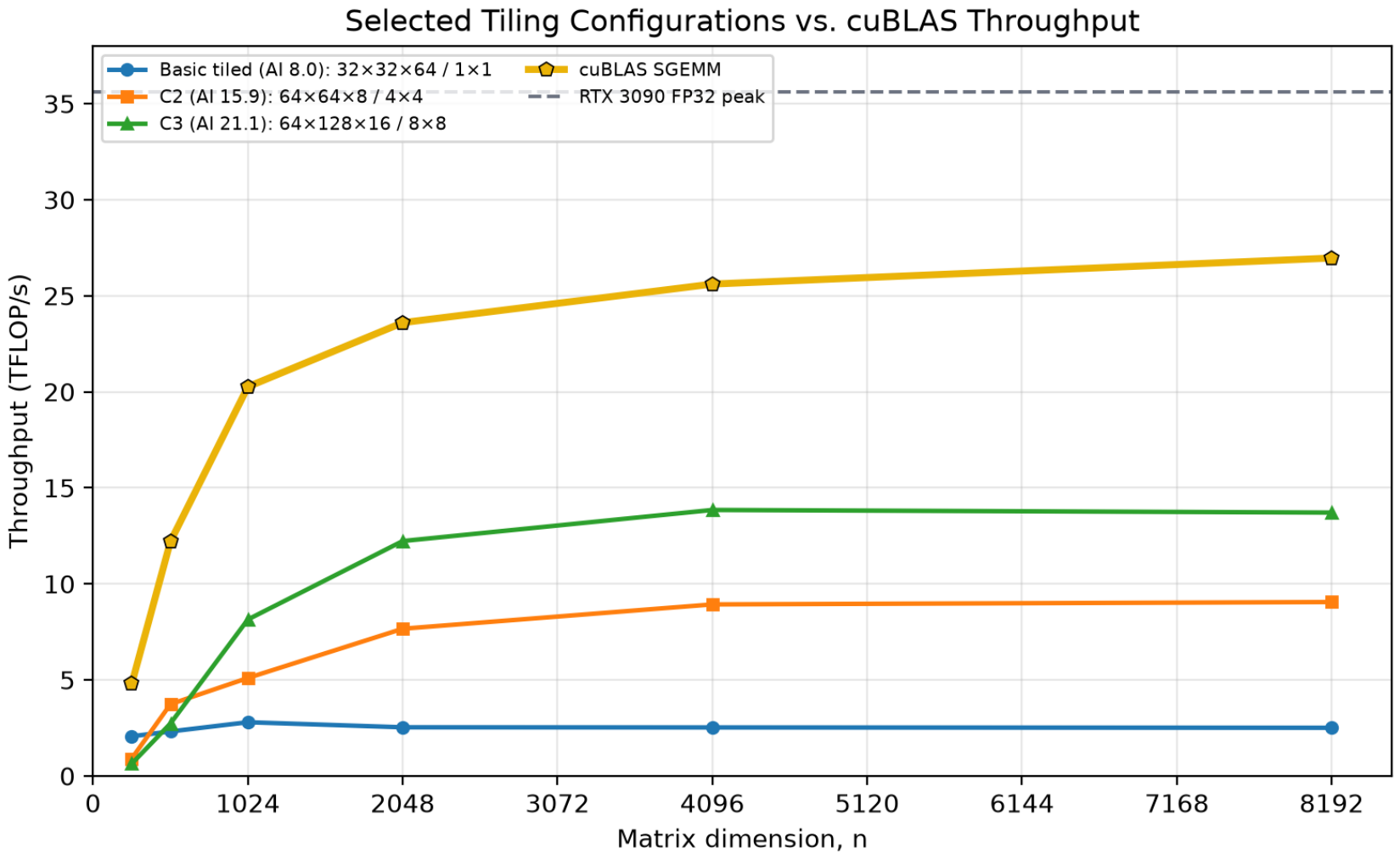
We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 26.96 TFLOP/s, while the highest-AI selected configuration (C3) reaches 13.71 TFLOP/s, or 50.8% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.8 ms versus 80.2 ms for C3, about 1.97× faster.

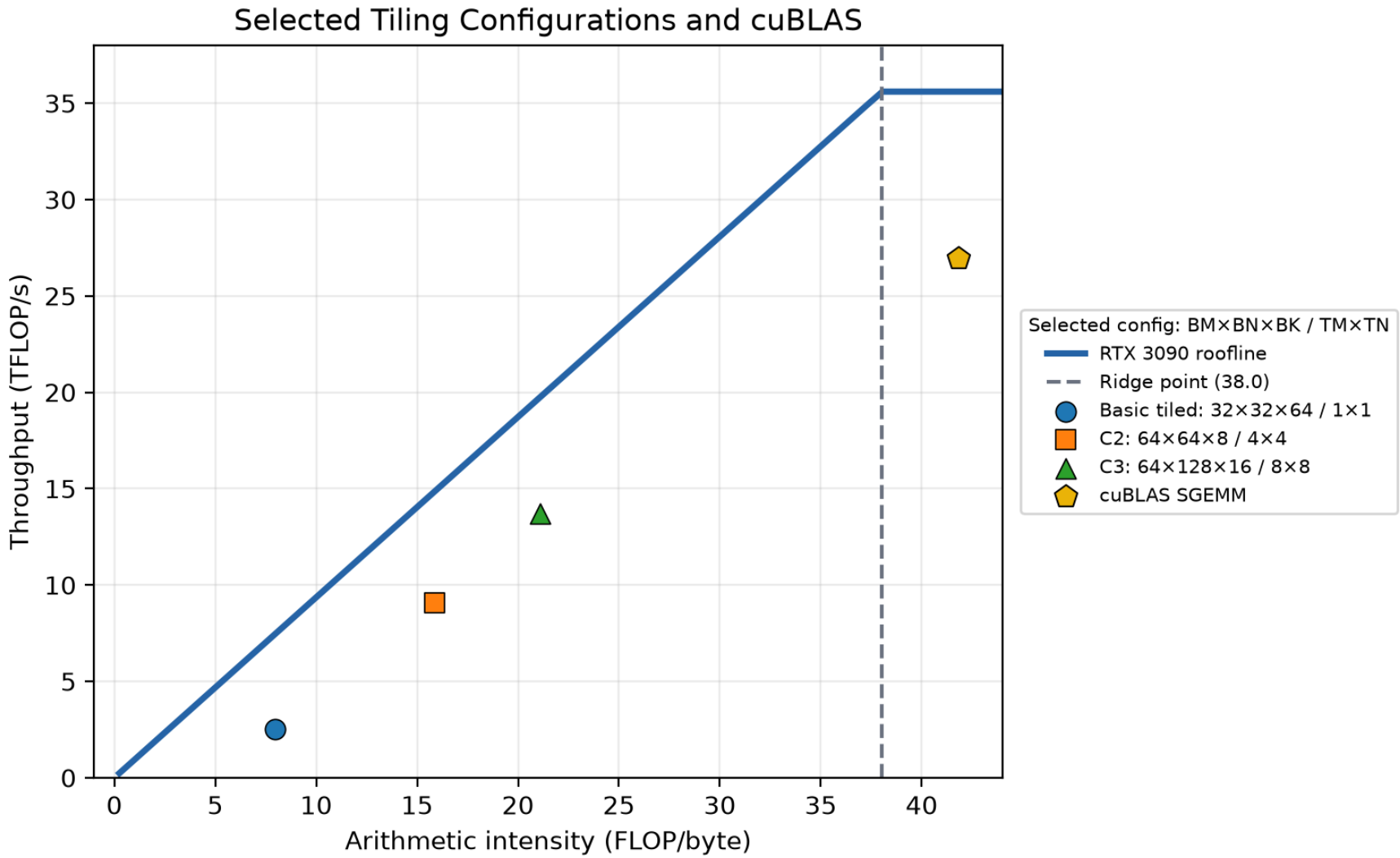
The remaining gap

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

cuBLAS remains the performance target



cuBLAS on the roofline



cuBLAS arithmetic intensity is estimated from the profiled 256×128 output-tile shape.

We've come a long way

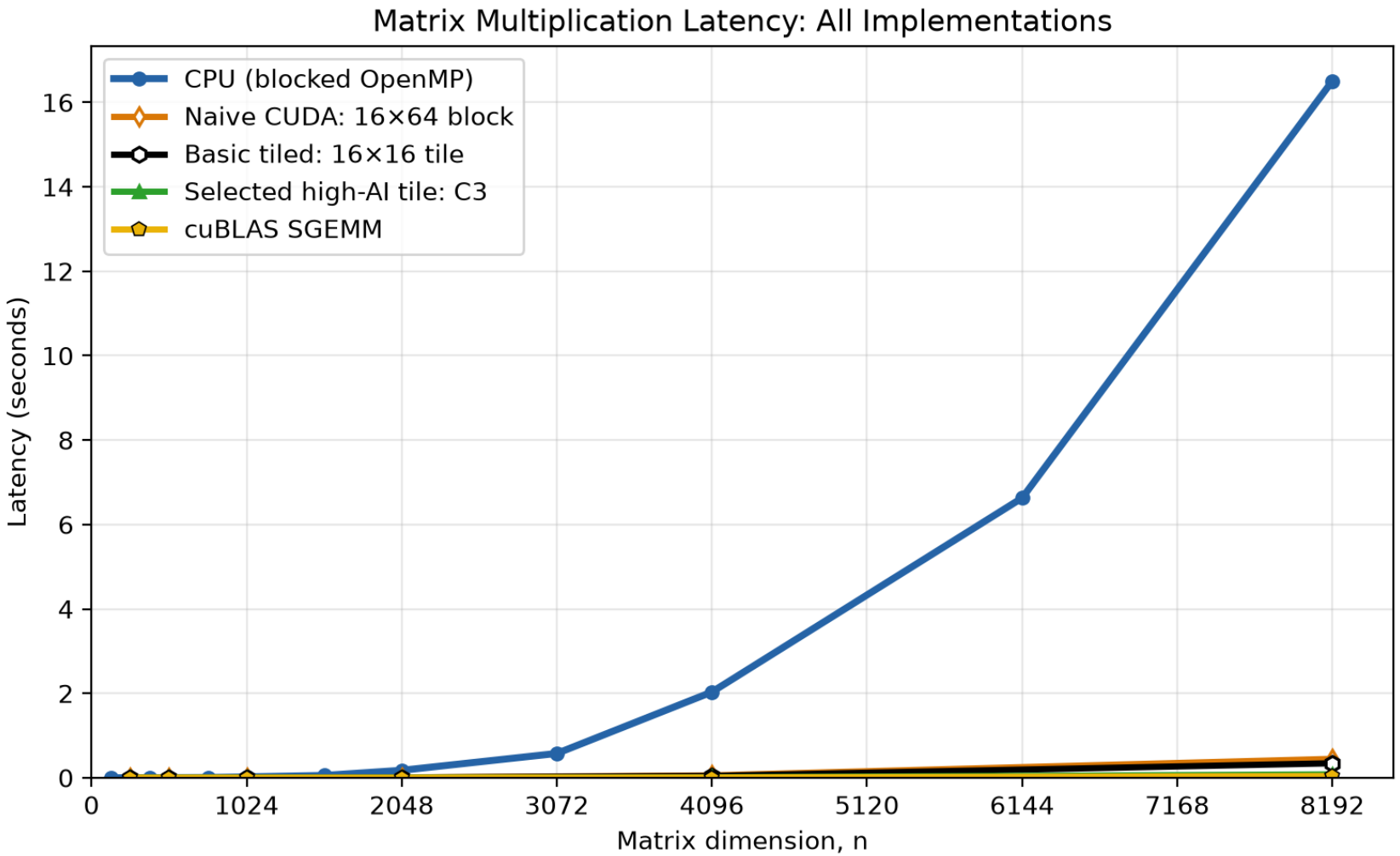


Table of Contents

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

More Kernel Optimizations for SGEMM

We have only scratched the surface of optimizing SGEMM. Other topics include:

1. **Memory coalescing:** arrange accesses so neighboring threads use neighboring addresses and require fewer memory transactions.
2. **Occupancy and register pressure:** balance per-thread resources against the number of warps available to hide latency.
3. **Shared-memory bank conflicts:** pad or rearrange tiles so threads in a warp do not serialize their accesses.
4. **Vectorized memory operations:** use aligned `float4` loads and stores to reduce instruction overhead.
5. **Warp tiling:** divide a block's output tile among warps to improve locality and register reuse.
6. **Autotuning:** search tile sizes and launch parameters for the target GPU instead of relying on one universal configuration.

Alternative Programming Models and DSLs

In production, writing raw CUDA is rarely worth the effort.

1. Triton: tiling is at the heart of GPU optimization, so that becomes our new atomic unit instead of threads. Thread-level optimizations are done by a JIT compiler.
2. CUTLASS, CuTe DSL, Layout Algebra: Manually handling indexing is cumbersome and error-prone. We can compose common patterns with an algebra that can be checked at compile time.

Kernel Fusion and Specialized Kernels

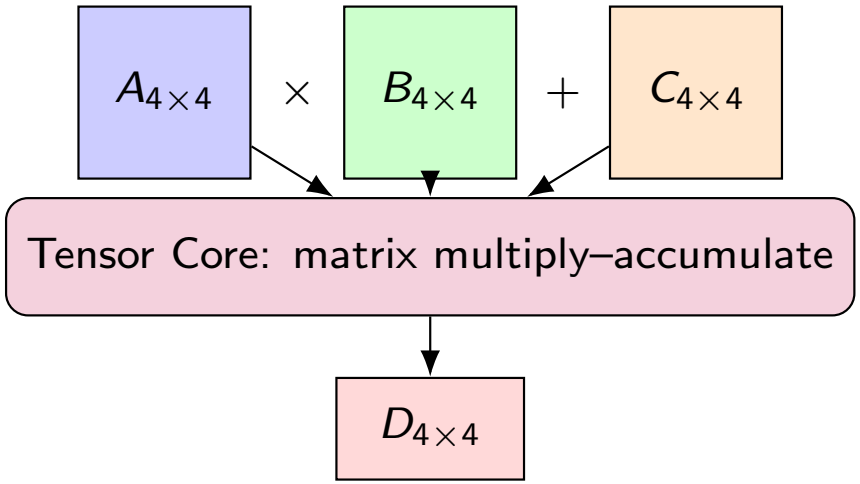
Kernels, be definition, have to read/write from DRAM. We can **fuse** kernels to increase performance.

1. Linear Layer + Nonlinearity = Fused MLP layer
2. FlashAttention series: complex fusion of the Attention Mechanism
3. Megakernels: taken to the extreme, we can fuse the operations necessary for the entire forward pass of a model into a **megakernel.**

Continuing GPU evolution

2017: Volta Tensor Cores

- Tensor Cores perform mixed-precision matrix multiply–accumulate: $D = AB + C$.
- The original Volta unit multiplies FP16 inputs and accumulates in FP16 or FP32.
- Matrix multiplication becomes an explicit hardware primitive, alongside general-purpose CUDA cores.



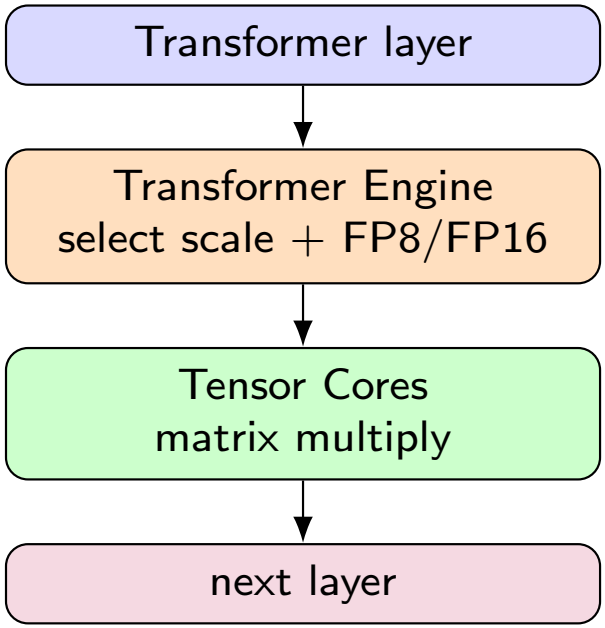
Operation shape shown for first-generation Volta Tensor Cores. Source:

NVIDIA, *Volta Architecture Whitepaper*, 2017.

Continuing GPU evolution

2020 onward: specialize around AI dataflow

- Ampere (2020) adds TF32 and structured sparsity support.
- Hopper (2022) adds FP8 Tensor Cores and a Transformer Engine that dynamically chooses FP8 or FP16 per layer.
- Blackwell (2024) adds more low-precision formats and a second-generation Transformer Engine. This is more than incremental scaling.



Source: NVIDIA, *Hopper Architecture In-Depth* (2022).

Continuing GPU evolution

2026: Vera Rubin, optimizing the rack

- The target is no longer an isolated GPU, but a complete rack-scale computing platform.
- Vera CPUs, Rubin GPUs, memory, NVLink, and networking are designed together to run one distributed AI workload.
- A high-bandwidth scale-up fabric lets many accelerators share model parameters and intermediate results.
- Performance now depends on communication, power, cooling, and software orchestration as well as matrix throughput.

Unit of optimization

Kernel

↓

GPU

↓

Server

↓

Rack-scale system

Source: NVIDIA, *Vera Rubin Architecture*, announced roadmap.

Alternative Accelerator Architectures

Big Picture

With respect to AI accelerators, GPUs are actually remarkably **general**. We can further specialize into AI-specific compute operations.

Alternative Accelerator Architectures

Big Picture

With respect to AI accelerators, GPUs are actually remarkably **general**. We can further specialize into AI-specific compute operations.

Alternative Accelerator Architectures

Big Picture

With respect to AI accelerators, GPUs are actually remarkably **general**. We can further specialize into AI-specific compute operations.

1. TPUs: Specializing into the SGEMM at the hardware level.

Alternative Accelerator Architectures

Big Picture

With respect to AI accelerators, GPUs are actually remarkably **general**. We can further specialize into AI-specific compute operations.

1. TPUs: Specializing into the SGEMM at the hardware level.
2. Etched SOHU: Specializing into the Transformer NN architecture at the hardware level.

Alternative Accelerator Architectures

Big Picture

With respect to AI accelerators, GPUs are actually remarkably **general**. We can further specialize into AI-specific compute operations.

1. TPUs: Specializing into the SGEMM at the hardware level.
2. Etched SOHU: Specializing into the Transformer NN architecture at the hardware level.
3. NPUs: Serving AI workloads in power and heat constrained environments (mobile, consumer electronics, edge devices, etc.).

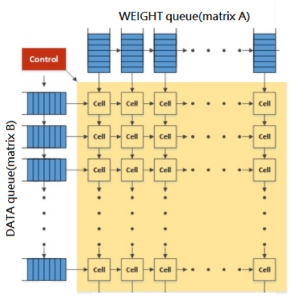
Alternative Accelerator Architectures

Big Picture

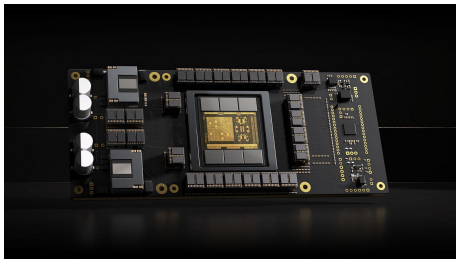
With respect to AI accelerators, GPUs are actually remarkably **general**. We can further specialize into AI-specific compute operations.

1. TPUs: Specializing into the SGEMM at the hardware level.
2. Etched SOHU: Specializing into the Transformer NN architecture at the hardware level.
3. NPUs: Serving AI workloads in power and heat constrained environments (mobile, consumer electronics, edge devices, etc.).
4. WSE (Wafer Scale Engine): Massive data bandwidth through multi-die wafers.

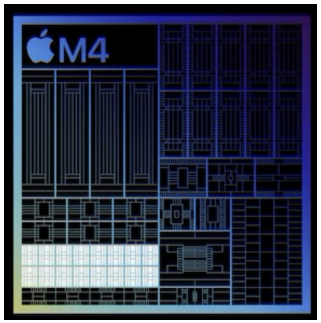
Takeaways



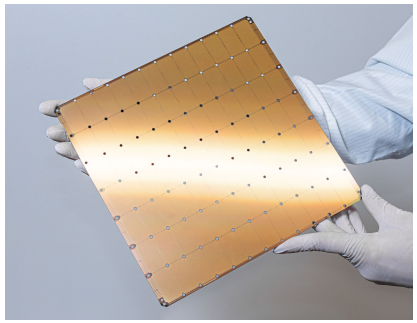
TPU systolic array



Etched Sohu



Apple M4 NPU



Cerebras WSE

There is no universally best architecture

Everything is a tradeoff. Optimizing for something always comes with a cost. **Workflow demands shape hardware evolution.**

Table of Contents

1. Why is Matrix Multiplication so important in AI?
2. Why GPUs? What's wrong with CPUs?
3. How did we get here?
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

Conclusion

1. **AI workloads expose parallel linear algebra.** Neural-network layers repeatedly reduce to matrix and tensor operations.
2. **GPUs already had the right foundation.** Graphics demanded parallel throughput and high memory bandwidth; unified shaders and CUDA made that hardware programmable.
3. **Data movement determines real performance.** Shared-memory tiling and register reuse move GEMM toward the compute limit.
4. **AI is now driving specialization.** Tensor Cores, lower precision, and new accelerators trade flexibility for throughput and efficiency.

Main takeaway

Graphics and AI both reward massive parallelism. The GPU succeeded by evolving from a graphics pipeline into a programmable matrix engine.

References: GPU evolution

- I. Buck et al. (2004), "Brook for GPUs: Stream Computing on Graphics Hardware."
- E. Lindholm et al. (2008), "NVIDIA Tesla: A Unified Graphics and Computing Architecture."
- NVIDIA (2009), *Fermi Compute Architecture Whitepaper*.
- NVIDIA (2017), *NVIDIA Tesla V100 GPU Architecture*.
- NVIDIA (2020), "NVIDIA Ampere Architecture In-Depth."
- NVIDIA (2022), "NVIDIA Hopper Architecture In-Depth."
- NVIDIA (2024), "NVIDIA Blackwell Architecture."
- NVIDIA, "NVIDIA Rubin Architecture."

References: CUDA and GEMM

- S. Boehm (2022), "How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance."
- S. Williams, A. Waterman, and D. Patterson (2009), "Roofline: An Insightful Visual Performance Model for Multicore Architectures."
- NVIDIA, *CUDA C++ Programming Guide*.
- NVIDIA, "GeForce RTX 3090 Family Specifications."
- NVIDIA, "CUDA Refresher: The CUDA Programming Model."
- NVIDIA, *cuBLAS Library Documentation*.
- Modal, "CUDA Programming Model," *GPU Glossary*.
- F. Sanglard (2020), "The CUDA Architecture."

References: alternative accelerators

- AMD, "AMD Instinct Accelerators."
- N. P. Jouppi et al. (2017), "In-Datacenter Performance Analysis of a Tensor Processing Unit."
- V. Sze et al. (2017), "Efficient Processing of Deep Neural Networks: A Tutorial and Survey."
- Cerebras, "The Wafer-Scale Engine."
- Etched (2024), "Introducing Sohu: The World's First Transformer ASIC."

Questions?